Machine Learning

(Basics and Linear Algebra)

Deepak Sharma

BSBE and DS&AI

IIT Roorkee

Regular vs ML algorithm

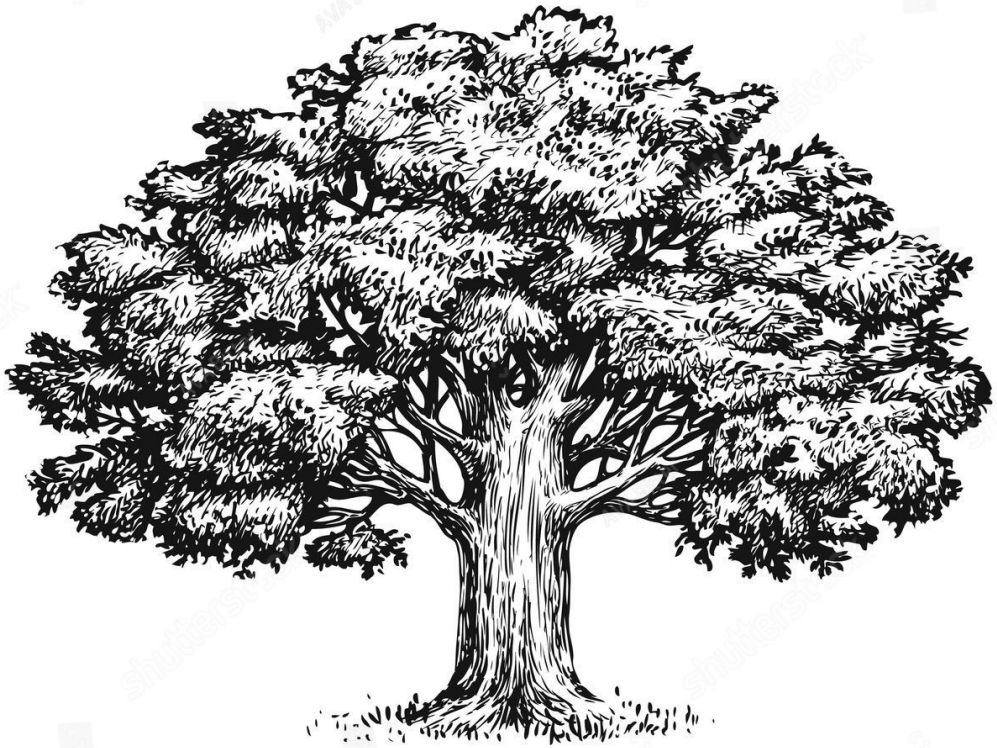
- A regular algorithm has fixed rules (step-by-step instructions) for doing a task.
- An ML algorithm makes it own rules (more the data better the algorithm/result).

Tree identification

Features

- *Brown trunk*
- *Brown branches*
- *Green leaves*

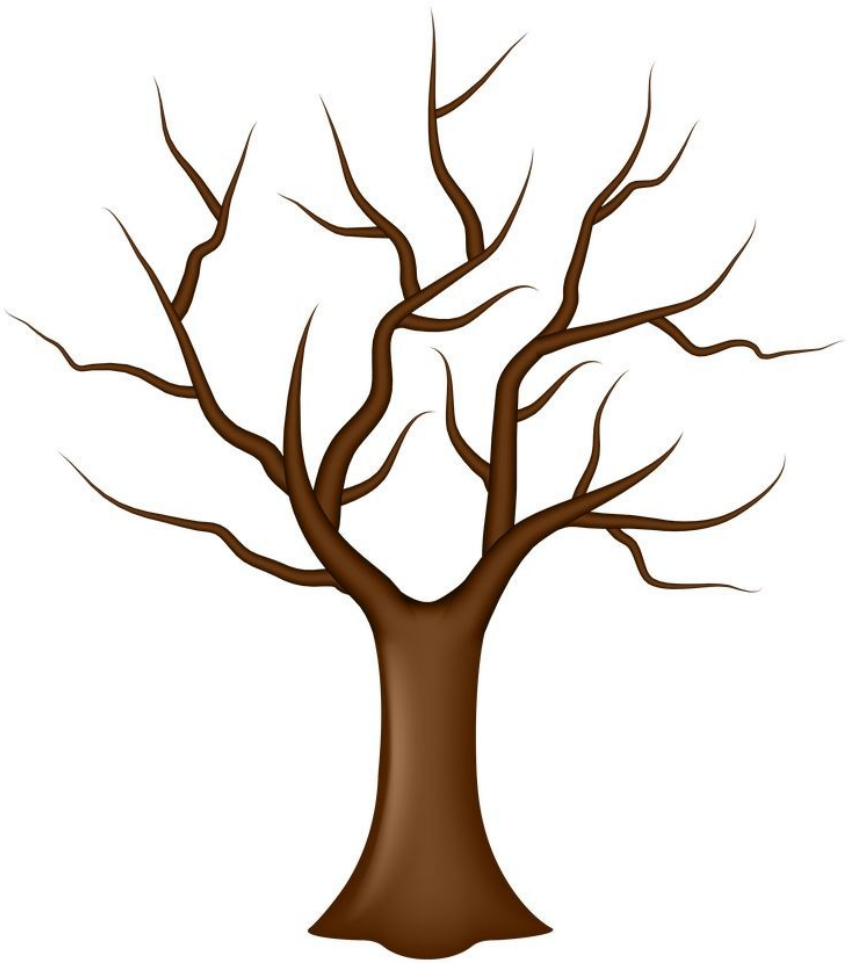
Tree identification



Features

- *Brown trunk*
- *Brown branches*
- *Green leaves*

Tree identification



Features

- Brown trunk
- Brown branches
- Green leaves

Tree identification

Features

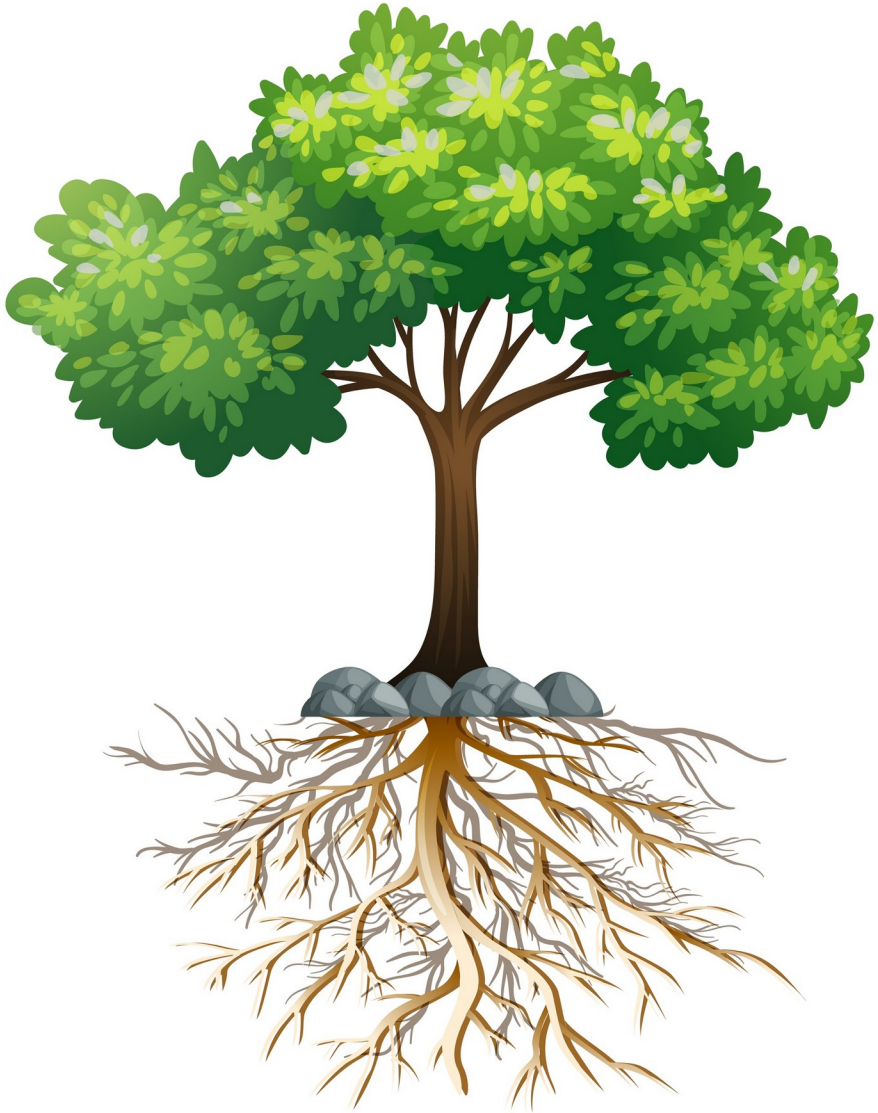
- Brown trunk
- Brown branches
- Green leaves

Tree identification

Features

- Brown trunk
- Brown branches
- Green leaves

Tree identification



Features

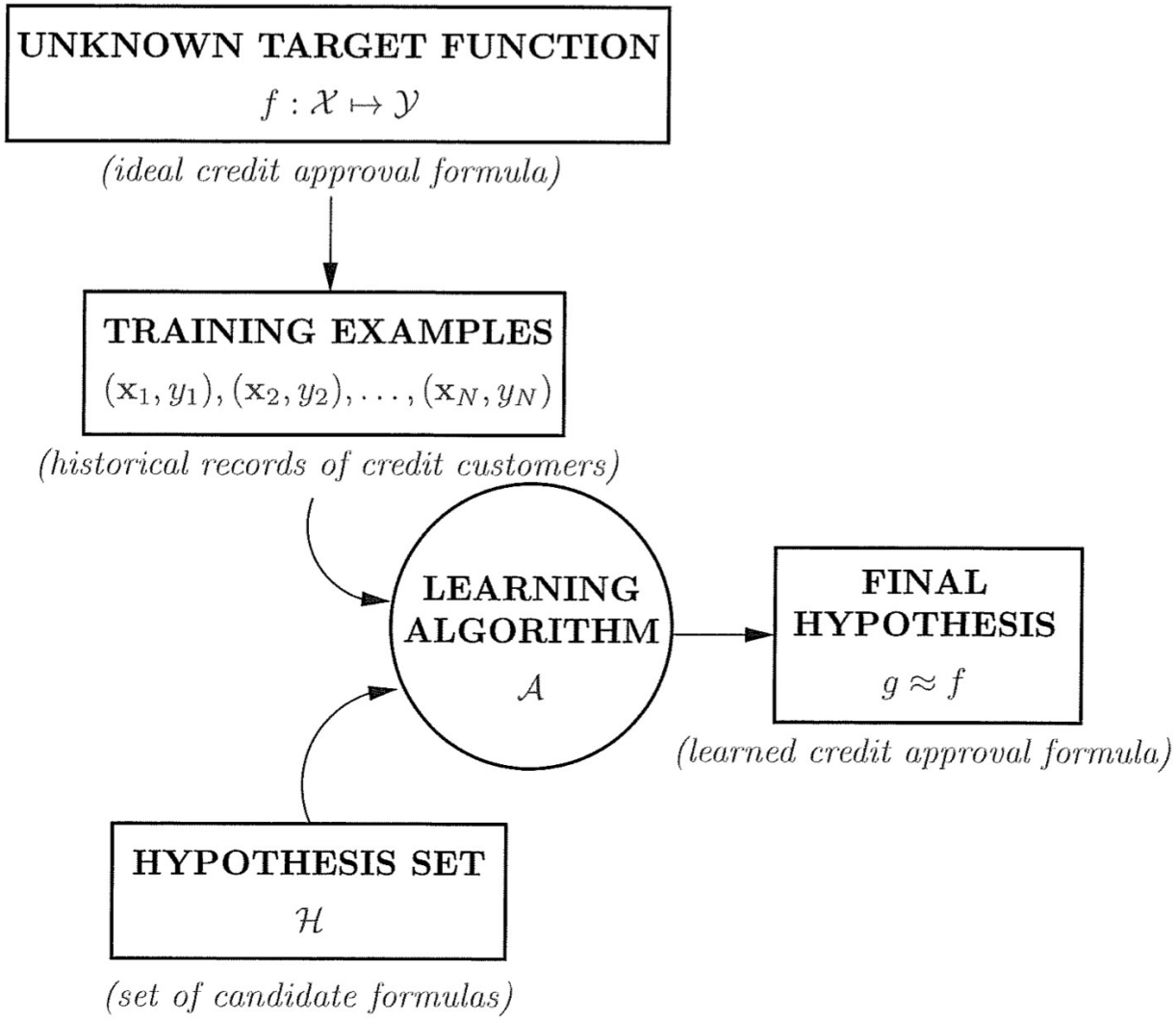
- ~~Brown trunk~~
- ~~Brown branches~~
- ~~Green leaves~~
- Roots

Tree identification

Features

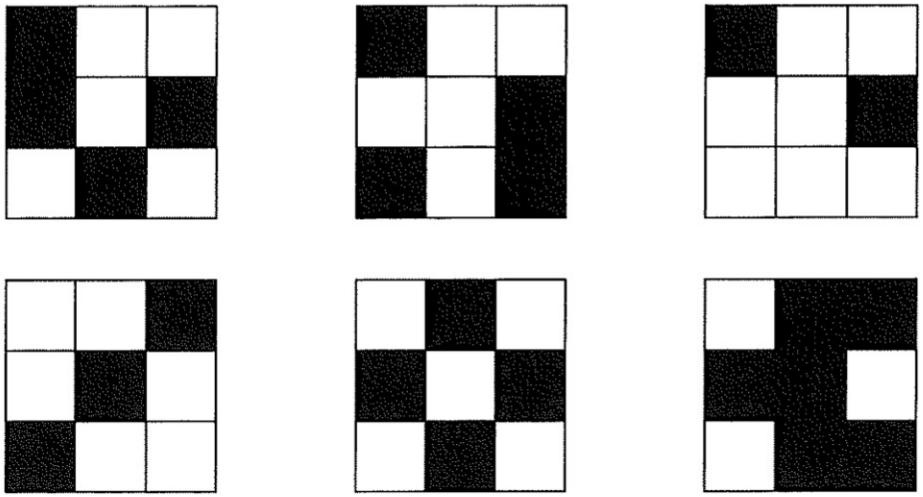
- ~~Brown trunk~~
- ~~Brown branches~~
- ~~Green leaves~~
- Roots
- *Modifying/Improving*

Basic setup of learning problem



Is learning feasible?

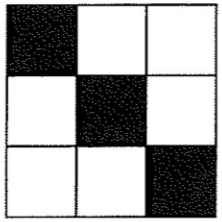
Training examples



$f = -1$

$f = +1$

Test input



$f = ?$

- *The difficulty we experienced in this simple problem is the rule, not the exception!*

Is learning feasible? (Concrete case)

Consider a Boolean target function over a three-dimensional input space $\mathcal{X} = \{0,1\}^3$.

We are given a data set $\mathcal{D}$ of five examples represented in the table below. We denote the binary output by $\circ/\bullet$ for visual clarity,

$\mathbf{x}_n$	y_n
0 0 0	$\circ$
0 0 1	$\bullet$
0 1 0	$\bullet$
0 1 1	$\circ$
1 0 0	$\bullet$

where $y_n = f(\mathbf{x}_n)$ for $n = 1, 2, 3, 4, 5$. The advantage of this simple Boolean case is that we can enumerate the entire input space (since there are only $2^3 = 8$ distinct input vectors), and we can enumerate the set of all possible target functions (since f is a Boolean function on 3 Boolean inputs, and there are only $2^{2^3} = 256$ distinct Boolean functions on 3 Boolean inputs).

You can think of f as a 8-bit vector
E.g., $f = [+1, -1, -1, -1, +1, +1, +1, -1]$.
So there are $2^8 = 256$ possible f's.

Is learning feasible? (Concrete case)

One 1's will give me •; Others give me ○

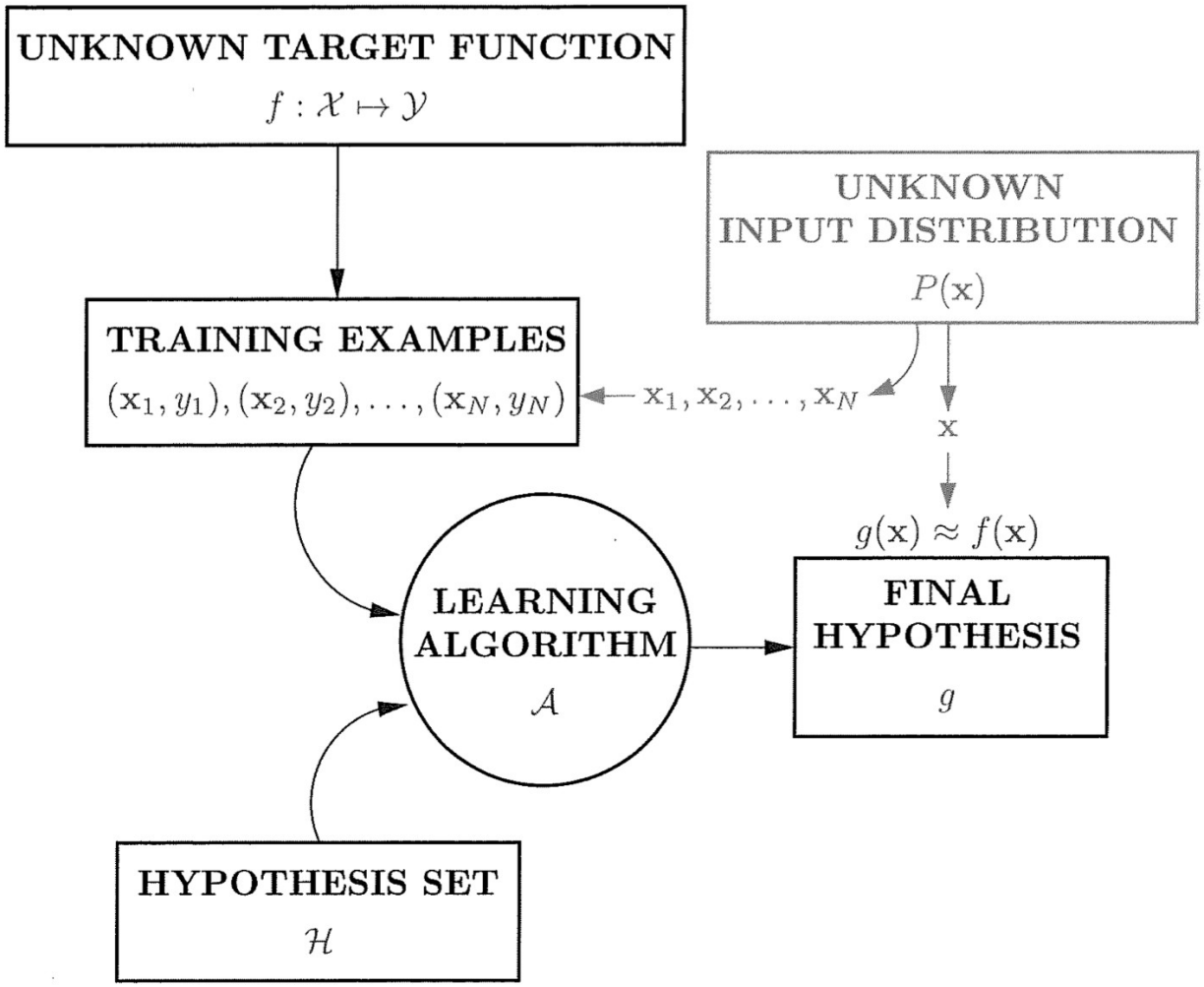
Odd numbers of 1's give me •

Even numbers of 1's give me ○

$\mathbf{x}$	y	g	f_1	f_2	f_3	f_4	f_5	f_6	f_7	f_8
0 0 0	○	○	○	○	○	○	○	○	○	○
0 0 1	•	•	•	•	•	•	•	•	•	•
0 1 0	•	•	•	•	•	•	•	•	•	•
0 1 1	○	○	○	○	○	○	○	○	○	○
1 0 0	•	•	•	•	•	•	•	•	•	•
1 0 1		?	○	○	○	○	•	•	•	•
1 1 0		?	○	○	•	•	○	○	•	•
1 1 1		?	○	•	○	•	○	•	○	•

- *We just don't know which one out of the eight to choose!*
- *So we haven't learned anything from the training data.*
- *Learning is infeasible!*

Power of probability (Rescue)

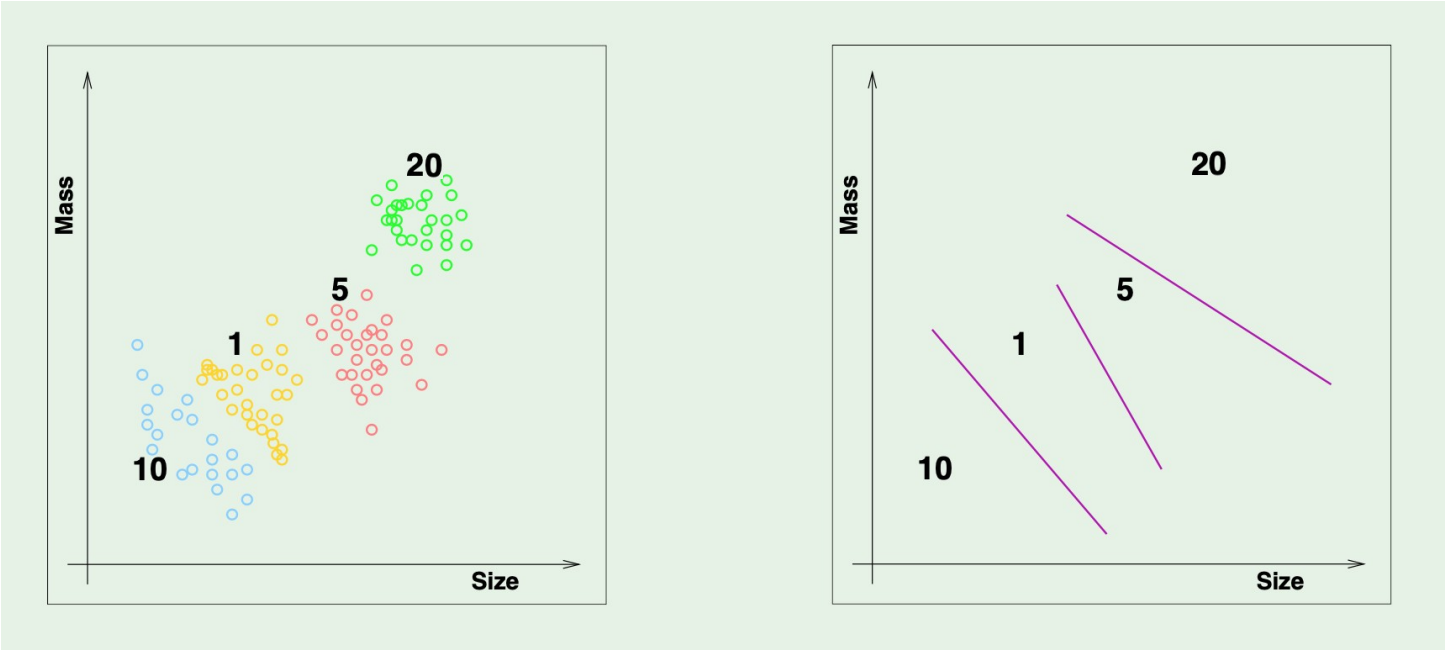


Basic premise of learning

- *Using a set of observations to uncover an underlying process*
- *Broad premise ⇒ many variations*
  - *Supervised Learning*
  - *Unsupervised Learning*
  - *Reinforcement Learning*

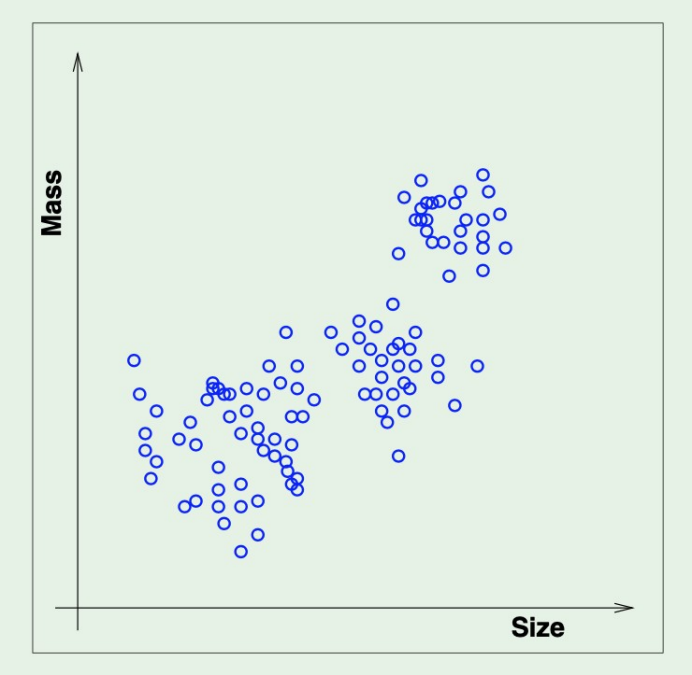
Supervised learning

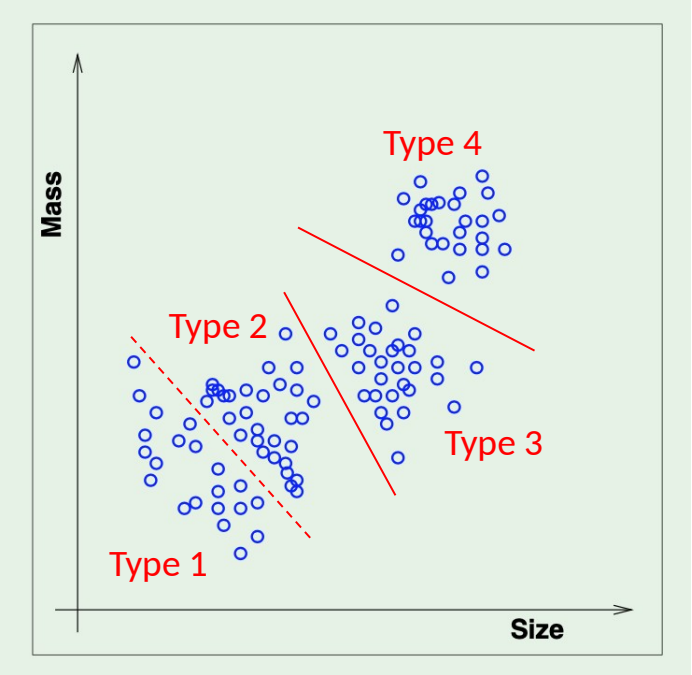
Eg. Coin recognition in vending machines



Unsupervised learning

Instead of (input, correct output) we get (input, ?)



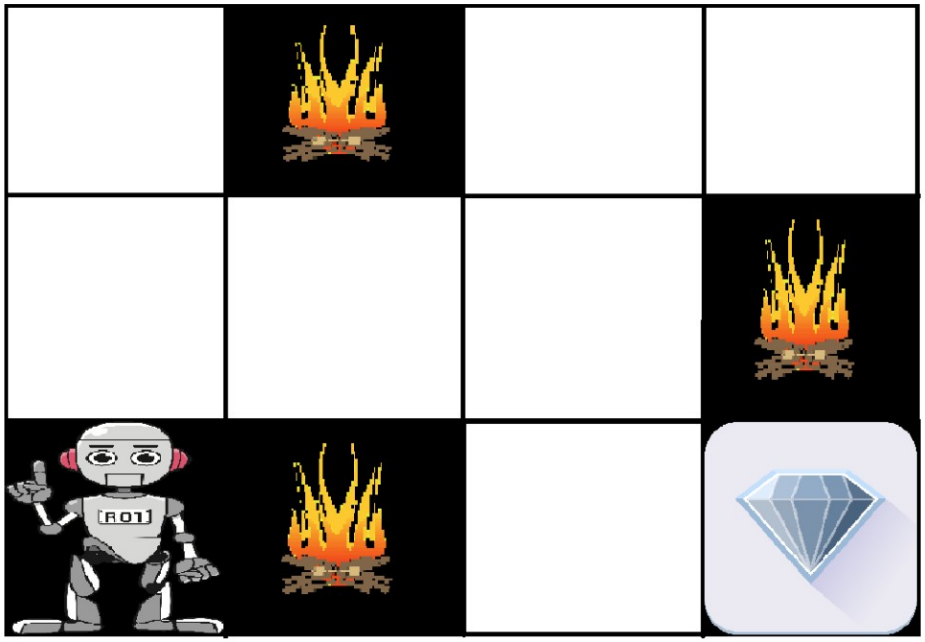


Reinforcement learning

Instead of (input, correct output)

we get (input, some output, grade for this output)

Eg. Navigating a maze [Agent, Rewards (+ve), Hurdles (-ve)]



Vector, Matrix, Tensor

Python — vi vector_tensor.py — 62×26

```
#!/Users/deepak/opt/anaconda3/bin/python3

####################################
# Scalar, Vector, Matrix, Tensor
####################################

import numpy as np

# Scalars can be represented simply as numerical variables
scalar = 9
print("\n",scalar)

# Vectors can be represented as one-dimensional ar
vector = np.array([ -3, -2, -1, 0, 1, 2, 3])
print("\n", vector)

# Matrices can be represented as two-dimensional a
matrix = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]
print("\n", matrix)

# Tensors can be represented as multi-dimensional
tensor = np.array([[[1, 2], [3, 4]], [[5, 6], [7,
print("\n", tensor, "\n")
~
~
"vector_tensor.py" 23L, 651C
```

Python — -bash — 62×18

```
(base) Deepaks-MacBook-Pro:Python deepak$ ./vector_tensor.py

 9

 [-3 -2 -1  0  1  2  3]

 [[1 2 3]
 [4 5 6]
 [7 8 9]]

 [[[1 2]
  [3 4]]

 [[5 6]
  [7 8]]]

(base) Deepaks-MacBook-Pro:Python deepak$
```

Basic concepts

- *Linear algebra provides a way of compactly representing and operating on sets of linear equations. For example:*

$$\begin{aligned} 4x_1 - 5x_2 &= -13 \\ -2x_1 + 3x_2 &= 9 \end{aligned}$$

- *In matrix notation, we can write the system more compactly as*

$$Ax = b$$

with

$$A = \begin{bmatrix} 4 & -5 \\ -2 & 3 \end{bmatrix}, \quad b = \begin{bmatrix} -13 \\ 9 \end{bmatrix}$$

Basic notations

Capital letter

- By $A \in \mathbb{R}^{m \times n}$ we denote a matrix with m rows and n columns, where the entries of A are real numbers.

Small letter

- By $x \in \mathbb{R}^n$, we denote a vector with n entries. By convention, an n-dimensional vector is often thought of as a matrix with n rows and 1 column, known as a ***column vector***. If we want to explicitly represent a ***row vector*** — a matrix with 1 row and n columns — we typically write x^T (here x^T denotes the transpose of x, which we will define shortly).

- The ith element of a vector x is denoted x_i:

$$x = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix}.$$

Basic notations

- We use the notation a_{ij} (or A_{ij}, $A_{i,j}$, etc) to denote the entry of A in the ith row and jth column:

$$A = \begin{bmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m1} & a_{m2} & \cdots & a_{mn} \end{bmatrix}.$$

- We denote the jth column of A by a_j or $A_{:,j}$:

$$A = \begin{bmatrix} | & | & & | \\ a_1 & a_2 & \cdots & a_n \\ | & | & & | \end{bmatrix}.$$

- We denote the ith row of A by a_i^T or $A_{i,:}$:

$$A = \begin{bmatrix} — & a_1^T & — \\ — & a_2^T & — \\ & \vdots & \\ — & a_m^T & — \end{bmatrix}.$$

Transpose

The ***transpose*** of a matrix results from "flipping" the rows and columns. Given a matrix $A \in \mathbb{R}^{m \times n}$, its transpose, written $A^T \in \mathbb{R}^{n \times m}$, is the $n \times m$ matrix whose entries are given by

$$(A^T)_{ij} = A_{ji}.$$

Properties

- $(A^T)^T = A$
- $(AB)^T = B^T A^T$
- $(A + B)^T = A^T + B^T$

Matrix multiplication

The product of two matrices $A \in \mathbb{R}^{m \times n}$ and $B \in \mathbb{R}^{n \times p}$ is the matrix

$$C = AB \in \mathbb{R}^{m \times p},$$

where

$$C_{ij} = \sum_{k=1}^{n} A_{ik} B_{kj}.$$

$$\begin{bmatrix} a_{11}, a_{12} \\ a_{21}, a_{22} \end{bmatrix} \times \begin{bmatrix} b_{11}, b_{12} \\ b_{21}, b_{22} \end{bmatrix}$$

$$\begin{bmatrix} a_{11} \times b_{11} + a_{12} \times b_{21} , a_{11} \times b_{12} + a_{12} \times b_{22} \\ a_{21} \times b_{11} + a_{22} \times b_{21} , a_{21} \times b_{12} + a_{22} \times b_{22} \end{bmatrix}$$

Matrix multiplication

Python — vi matrices_product_np.py — 53×30

```
#!/Users/deepak/opt/anaconda3/bin/python3

###################################
# Multiply two matrices
###################################

import numpy as np

X = [[1,2,3],
     [4,5,6],
     [7,8,9]]

Y = [[9,8,7],
     [6,5,4],
     [3,2,1]]

print("\nMatrix X :")
print(X, "\n")
print("Matrix Y :")
print(Y, "\n")

# computing product
result = np.dot(X, Y)

# printing the result
print("The matrix multiplication is :")
print(result,"\n")
~
~
"matrices_product_np.py" 27L, 460C
```

Python — -bash — 67×15

```
(base) Deepaks-MacBook-Pro:Python deepak$ ./matrices_product_np.py

Matrix X :
[[1, 2, 3], [4, 5, 6], [7, 8, 9]]

Matrix Y :
[[9, 8, 7], [6, 5, 4], [3, 2, 1]]

The matrix multiplication is :
[[ 30  24  18]
 [ 84  69  54]
 [138 114  90]]

(base) Deepaks-MacBook-Pro:Python deepak$
```

Vector-vector product

Given two vectors $x, y \in \mathbb{R}^n$, the quantity $x^T y$, sometimes called the ***inner product*** or ***dot product*** of the vectors, is a real number given by

$$x^T y \in \mathbb{R} = \begin{bmatrix} x_1 & x_2 & \cdots & x_n \end{bmatrix} \begin{bmatrix} y_1 \\ x_2 \\ \vdots \\ y_n \end{bmatrix} = \sum_{i=1}^{n} x_i y_i.$$

$1 \times n \qquad n \times 1 \qquad 1 \times 1$

$$x^T y = y^T x$$

Given vectors $x \in \mathbb{R}^m$, $y \in \mathbb{R}^n$ (not necessarily of the same size), $xy^T \in \mathbb{R}^{m \times n}$ is called the ***outer product*** of the vectors. It is a matrix whose entries are given by $(xy^T)_{ij} = x_i y_j$, i.e.,

$$xy^T \in \mathbb{R}^{m \times n} = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_m \end{bmatrix} \begin{bmatrix} y_1 & y_2 & \cdots & y_n \end{bmatrix} = \begin{bmatrix} x_1 y_1 & x_1 y_2 & \cdots & x_1 y_n \\ x_2 y_1 & x_2 y_2 & \cdots & x_2 y_n \\ \vdots & \vdots & \ddots & \vdots \\ x_m y_1 & x_m y_2 & \cdots & x_m y_n \end{bmatrix}.$$

$m \times 1 \qquad 1 \times n \qquad m \times n$

Identity and Diagonal Matrix

The ***identity matrix***, denoted $I \in \mathbb{R}^{n \times n}$, is a square matrix with ones on the diagonal and zeros everywhere else. That is,

$$I_{ij} = \begin{cases} 1 & i = j \\ 0 & i \neq j \end{cases}$$

$$\begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

It has the property that for all $A \in \mathbb{R}^{m \times n}$,

$$AI = A = IA.$$

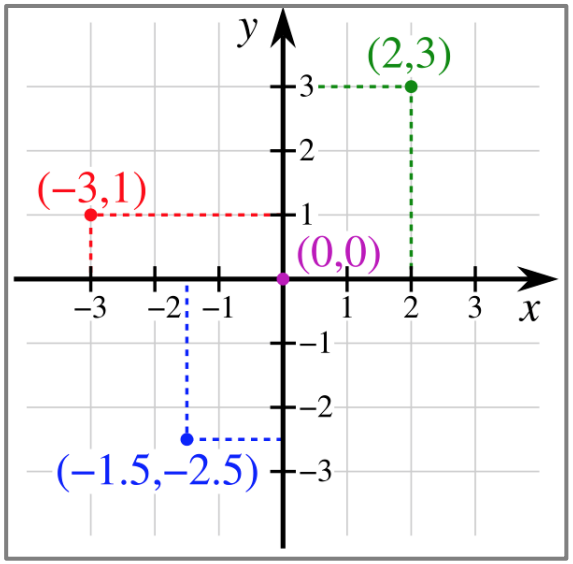
A ***diagonal matrix*** is a matrix where all non-diagonal elements are 0. This is typically denoted $D = \text{diag}(d_1, d_2, \ldots, d_n)$, with

$$D_{ij} = \begin{cases} d_i & i = j \\ 0 & i \neq j \end{cases}$$

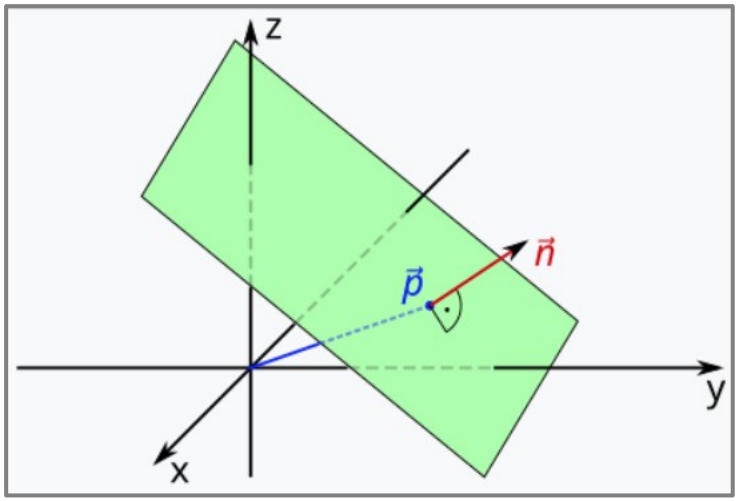
$$\begin{bmatrix} 6 & 0 & 0 \\ 0 & 5 & 0 \\ 0 & 0 & 4 \end{bmatrix}$$

Euclidean plane

- *Two dimension:* It is a geometric space in which two real numbers are required to determine the position of each point.

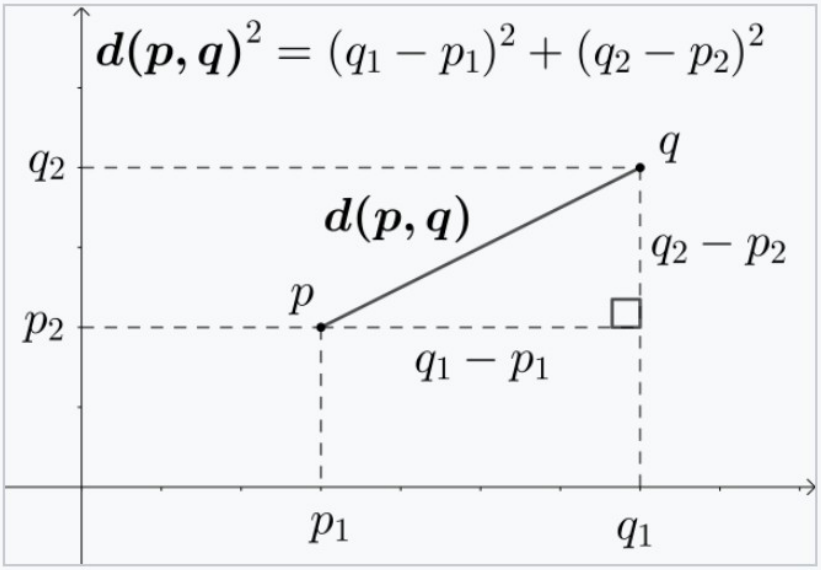


- *Three dimension:* a plane is a flat two-dimensional surface that extends indefinitely.



Euclidean distance

- *The distance between two points in Euclidean space is the length of the line segment between them.*



Using the Pythagorean theorem to compute two-dimensional Euclidean distance

Norm

A ***norm*** of a vector $\|x\|$ is informally a measure of the "length" of the vector.

The *Euclidean norm* or *length* of a vector $\mathbf{a} \in \mathbb{R}^m$ is defined as

L_2-norm

$$\|\mathbf{a}\| = \sqrt{\mathbf{a}^T\mathbf{a}} = \sqrt{a_1^2 + a_2^2 + \cdots + a_m^2} = \sqrt{\sum_{i=1}^{m} a_i^2}$$

The Euclidean norm is a special case of a general class of norms, known as *L_p-norm*, defined as

$$\|\mathbf{a}\|_p = \left(|a_1|^p + |a_2|^p + \cdots + |a_m|^p\right)^{\frac{1}{p}} = \left(\sum_{i=1}^{m} |a_i|^p\right)^{\frac{1}{p}} \qquad p \neq 0$$

Norm

Python — vi norm.py — 50×14

```
#!/Users/deepak/opt/anaconda3/bin/python3

#####################################
# L2 norm
#####################################

import numpy as np

x = np.array([ -3, -2, -1, 0, 1, 2, 3])
a = np.linalg.norm(x)
print("\n", a, "\n")

~
"norm.py" 12L, 232C
```

Inverse

The ***inverse*** of a square matrix $A \in \mathbb{R}^{n \times n}$ is denoted A^{-1}, and is the unique matrix such that

$$A^{-1}A = I = AA^{-1}.$$

$$\mathbf{A}^{-1} = \frac{1}{\det(\mathbf{A})}\mathbf{C}^{\mathsf{T}}.$$

Cofactor matrix

Note that not all matrices have inverses. Non-square matrices, for example, do not have inverses by definition. However, for some square matrices A, it may still be the case that A^{-1} may not exist.

Properties

- $(A^{-1})^{-1} = A$
- $(AB)^{-1} = B^{-1}A^{-1}$
- $(A^{-1})^T = (A^T)^{-1}$. For this reason this matrix is often denoted A^{-T}.

Determinant

The ***determinant*** of a square matrix $A \in \mathbb{R}^{n\times n}$, is a function $\det : \mathbb{R}^{n\times n} \to \mathbb{R}$, and is denoted $|A|$ or $\det A$

The absolute value of the determinant of A, it turns out, is a measure of the "volume" of the set S.[2]

For example, consider the 2×2 matrix,

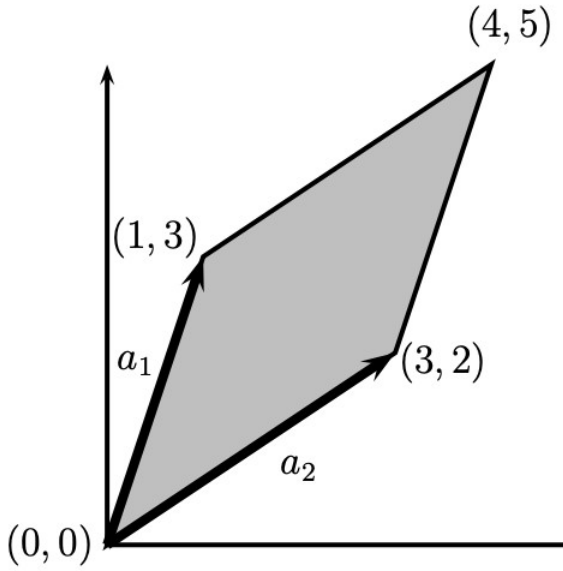
$$A = \begin{bmatrix} 1 & 3 \\ 3 & 2 \end{bmatrix}.$$

Here, the rows of the matrix are

$$a_1 = \begin{bmatrix} 1 \\ 3 \end{bmatrix} \quad a_2 = \begin{bmatrix} 3 \\ 2 \end{bmatrix}.$$

$$\boxed{|\det A| = 7}$$

Area of a parallelogram
3D: parallelepiped
n-dimensional parallelotope



Transformation matrix

- *A matrix that changes both direction and scale of the vector.*

$$TA = B$$

(where T is Transformation Matrix)

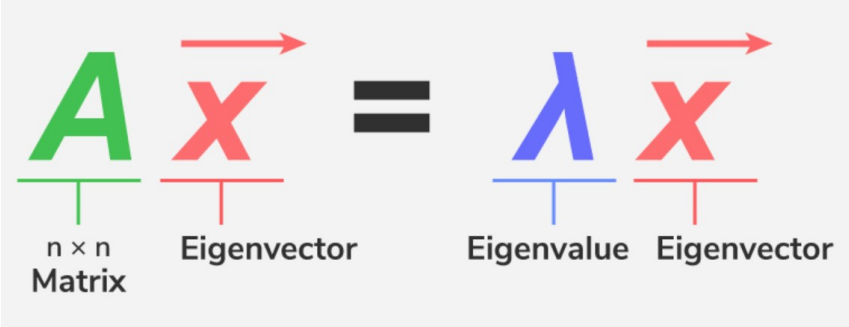
$$\begin{bmatrix} a & b \\ c & d \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} x' \\ y' \end{bmatrix}$$

Eigenvalues and Eigenvectors

Given a square matrix $A \in \mathbb{R}^{n \times n}$, we say that $\lambda \in \mathbb{C}$ is an ***eigenvalue*** of A and $x \in \mathbb{C}^n$ is the corresponding ***eigenvector***[3] if

$$\underset{(n \times n)}{A}\underset{(n \times 1)}{x} = \underset{(n \times 1)}{\lambda x}, \quad x \neq 0.$$

Intuitively, this definition means that multiplying A by the vector x results in a new vector that points in the same direction as x, but scaled by a factor λ.

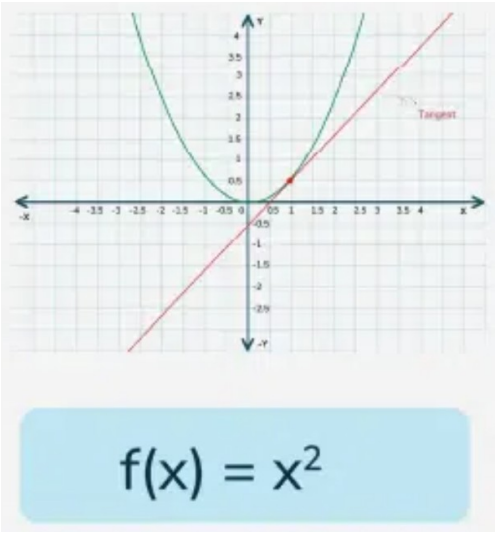


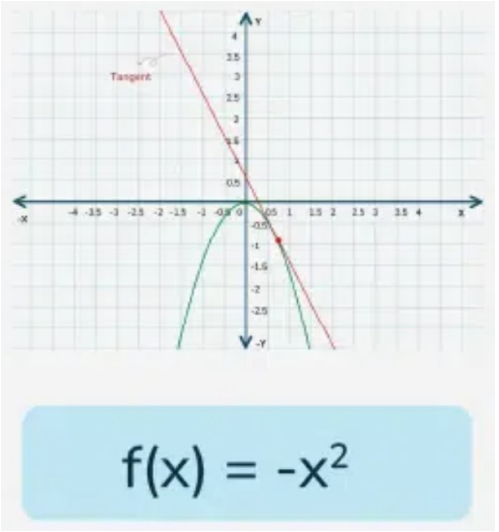
$$\begin{bmatrix} 1 & 2 \\ 5 & 4 \end{bmatrix} \begin{bmatrix} 2 \\ 5 \end{bmatrix} = 6 \begin{bmatrix} 2 \\ 5 \end{bmatrix}$$

$$\begin{bmatrix} 1 & 2 \\ 5 & 4 \end{bmatrix} \begin{bmatrix} 1 \\ -1 \end{bmatrix} = -1 \begin{bmatrix} 1 \\ -1 \end{bmatrix}$$

Concave and convex functions

- Convex and concave functions are functions with different curvature of graphs.
- A **convex function** curves upwards, meaning that any line segment connecting two points on the curve will lie above or on the graph; tangent line lies below the graph of the function.
- On the other hand, a **concave function** curves downwards, with any line segment connecting two points on the graph lying below or on the curve; tangent line lies above the curve.





Concave and convex functions

$f(x) = x^2$

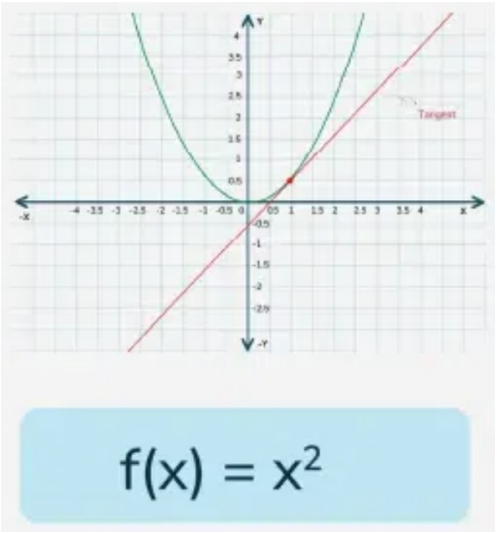
$f'(x) = 2x$

$f'(x) = 0$ *(Minima of function)*

$2x = 0$ or $x = 0$

$f''(x) = 2$

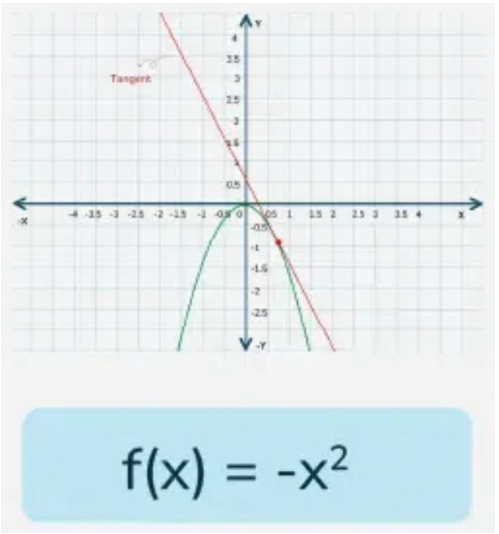
$f''(x) > 0$ *(Convex function)*



$f(x) = -x^2$

$f''(x) = -2$

$f''(x) < 0$ *(Concave function)*



Partial derivatives

- *A partial derivative of a function of several variables is its derivative with respect to one of those variables, with the others held constant.*

$$f(x,y) = f_y(x) = x^2 + xy + y^2$$

$$\frac{\partial f}{\partial x}(x,y) = 2x + y$$

Matrix calculus

Suppose that $f : \mathbb{R}^{m \times n} \to \mathbb{R}$ is a function that takes as input a matrix A of size $m \times n$ and returns a real value. Then the ***gradient*** of f (with respect to $A \in \mathbb{R}^{m \times n}$) is the matrix of partial derivatives, defined as:

$$\nabla_A f(A) \in \mathbb{R}^{m \times n} = \begin{bmatrix} \frac{\partial f(A)}{\partial A_{11}} & \frac{\partial f(A)}{\partial A_{12}} & \cdots & \frac{\partial f(A)}{\partial A_{1n}} \\ \frac{\partial f(A)}{\partial A_{21}} & \frac{\partial f(A)}{\partial A_{22}} & \cdots & \frac{\partial f(A)}{\partial A_{2n}} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial f(A)}{\partial A_{m1}} & \frac{\partial f(A)}{\partial A_{m2}} & \cdots & \frac{\partial f(A)}{\partial A_{mn}} \end{bmatrix}$$

i.e., an $m \times n$ matrix with

$$(\nabla_A f(A))_{ij} = \frac{\partial f(A)}{\partial A_{ij}}.$$

Note that the size of $\nabla_A f(A)$ is always the same as the size of A. So if, in particular, A is just a vector $x \in \mathbb{R}^n$,

$$\nabla_x f(x) = \begin{bmatrix} \frac{\partial f(x)}{\partial x_1} \\ \frac{\partial f(x)}{\partial x_2} \\ \vdots \\ \frac{\partial f(x)}{\partial x_n} \end{bmatrix}.$$

Machine Learning

(Probability)

Deepak Sharma
BSBE and DS&AI
IIT Roorkee

Probability is interesting

- *Three ants are moving from the vertices of a triangle what is the probability that no two ants will meet?*

1/4

Definition

- **Sample space** Ω: The set of all the outcomes of a random experiment.
- **Set of events** (or **event space**) $\mathcal{F}$: A set whose elements $A \in \mathcal{F}$ (called **events**) are subsets of Ω (i.e., $A \subseteq \Omega$ is a collection of possible outcomes of an experiment).
- **Probability measure**: A function $P : \mathcal{F} \rightarrow \mathbb{R}$ that satisfies the following properties,
  - $P(A) \geq 0$, for all $A \in \mathcal{F}$
  - $P(\Omega) = 1$

Definition

(1) The probability of an event E, denoted by $Pr(E)$, always satisfies $0 \le Pr(E) \le 1$.

(2) If outcomes A and B are two events that cannot both happen at the same time, then $Pr(A \text{ or } B \text{ occurs}) = Pr(A) + Pr(B)$.

The symbol { } is used as shorthand for the phrase "the event."

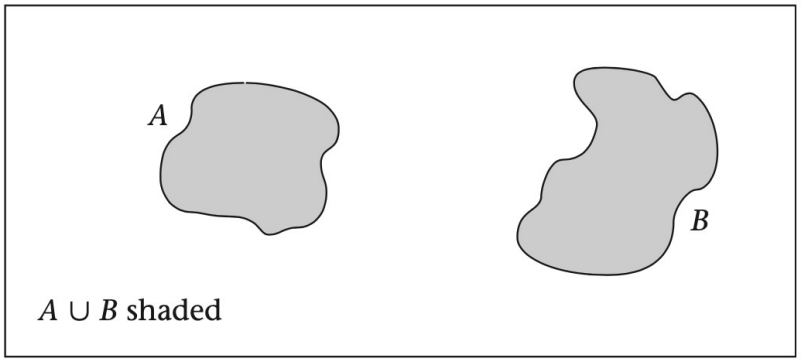
Hypertension Let A be the event that a person has normotensive diastolic blood-pressure (DBP) readings (DBP < 90), and let B be the event that a person has borderline DBP readings ($90 \le$ DBP < 95). Suppose that $Pr(A) = .7$, and $Pr(B) = .1$. Let Z be the event that a person has a DBP < 95. Then

$$Pr(Z) = Pr(A) + Pr(B) = .8$$

Notation

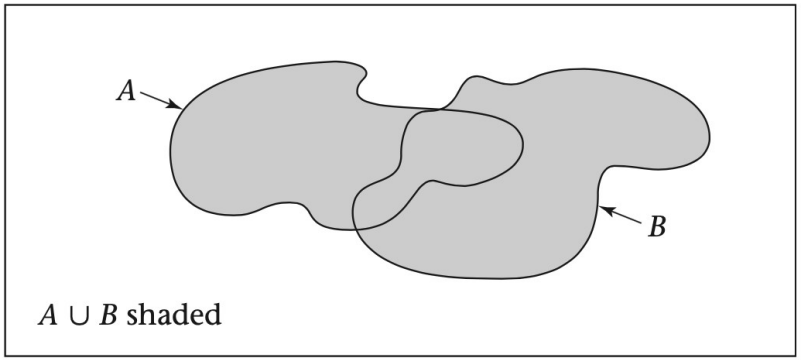
$A \cup B$ is the event that either A or B occurs, or they both occur.

Two events A and B are **mutually exclusive** if they cannot both happen at the same time.



(a)

Mutually exclusive

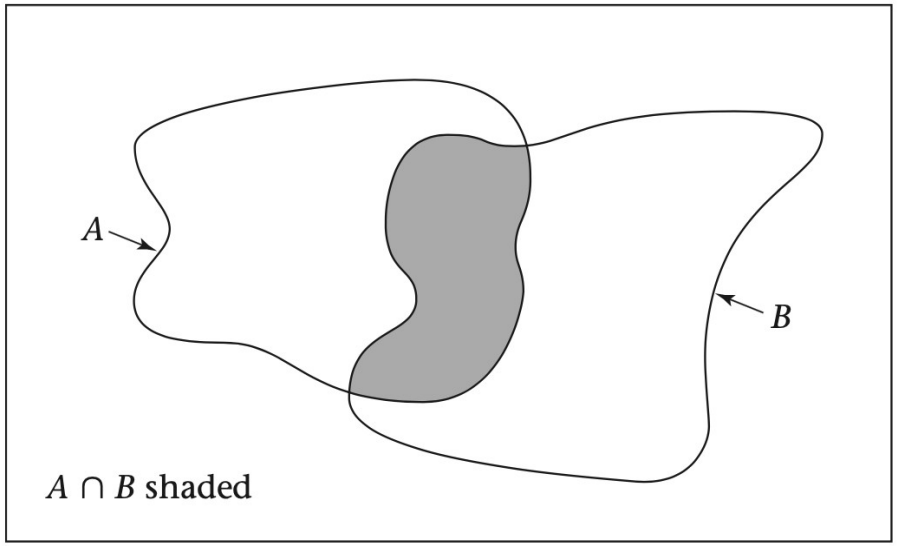


(b)

Not mutually exclusive

Notation

$A \cap B$ is the event that both A and B occur simultaneously.



Random variables

A **random variable** is a function that assigns numeric values to different events in a sample space.

A random variable for which there exists a discrete set of numeric values is a **discrete random variable.**

A random variable whose possible values cannot be enumerated is a **continuous random variable.**

Hypertension Suppose a physician agrees to use a new antihypertensive drug on a trial basis on the first four untreated hypertensives she encounters in her practice, before deciding whether to adopt the drug for routine use. Let X = the number of patients of four who are brought under control. Then X is a discrete random variable, which takes on the values 0, 1, 2, 3, 4.

PMF (Probability distribution)

A **probability-mass function** is a mathematical relationship, or rule, that assigns to any possible value r of a discrete random variable X the probability $Pr(X = r)$. This assignment is made for all values r that have positive probability. The probability-mass function is sometimes also called a **probability distribution.**

Probability-mass function for the hypertension-control example

$Pr(X = r)$	.008	.076	.265	.411	.240
r	0	1	2	3	4

$$.008 + .076 + .265 + .411 + .240 = 1$$

Expected value

The **expected value of a discrete random variable** is defined as

$$E(X) \equiv \mu = \sum_{i=1}^{R} x_i Pr(X = x_i)$$

Multinoulli distribution: >2 outcomes

Bernoulli distribution: 2 outcomes (Y/N or H/T; 0,1)

where the x_i's are the values the random variable assumes with positive probability.

Hypertension Find the expected value for the random variable

$$E(X) = 0(.008) + 1(.076) + 2(.265) + 3(.411) + 4(.240) = 2.80$$

Thus on average about 2.8 hypertensives would be expected to be brought under control for every 4 who are treated.

Poisson distribution

- *It is perhaps the second most frequently used discrete distribution after the binomial distribution.*
- *This distribution is usually associated with rare events.*

The probability of k events occurring in a time period t for a Poisson random variable with parameter λ is

$$Pr(X = k) = e^{-\mu}\mu^k/k!, \quad k = 0,1,2,\ldots$$

where $\mu = \lambda t$ and e is approximately 2.71828.

λ represents expected number of events per unit time (Expected value)
μ represents expected number of events over time period t

$$E(X) = \sum_{k=1}^{R} k\,Pr(X = k)$$

For a Poisson distribution with parameter μ, the mean and variance are both equal to μ.

Poisson distribution

Infectious Disease Consider the typhoid-fever example. Suppose the number of deaths from typhoid fever over a 1-year period is Poisson distributed with parameter $\mu = 4.6$. What is the probability distribution of the number of deaths over a 6-month period? A 3-month period?

Let X = the number of deaths in 6 months. Because $\mu = 4.6$, $t = 1$ year, it follows that $\lambda = 4.6$ deaths per year. For a 6-month period we have $\lambda = 4.6$ deaths per year, $t = .5$ year. Thus $\mu = \lambda t = 2.3$. Therefore,

$$Pr(X = 0) = e^{-2.3} = .100$$

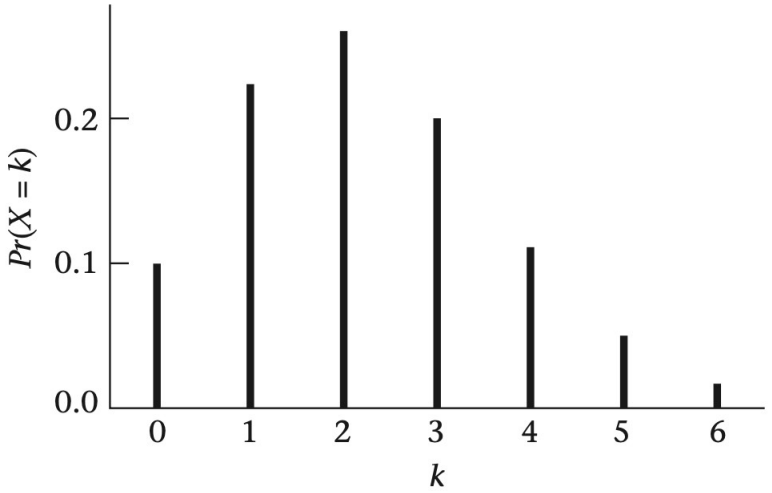
$$Pr(X = 1) = \frac{2.3}{1!} e^{-2.3} = .231$$

$$Pr(X = 2) = \frac{2.3^2}{2!} e^{-2.3} = .265$$

$$Pr(X = 3) = \frac{2.3^3}{3!} e^{-2.3} = .203$$

$$Pr(X = 4) = \frac{2.3^4}{4!} e^{-2.3} = .117$$

$$Pr(X = 5) = \frac{2.3^5}{5!} e^{-2.3} = .054$$



$$Pr(X \geq 6) = 1 - (.100 + .231 + .265 + .203 + .117 + .054) = .030$$

Normal/Gaussian Distribution (CPD)

The **normal distribution** is defined by its pdf, which is given as

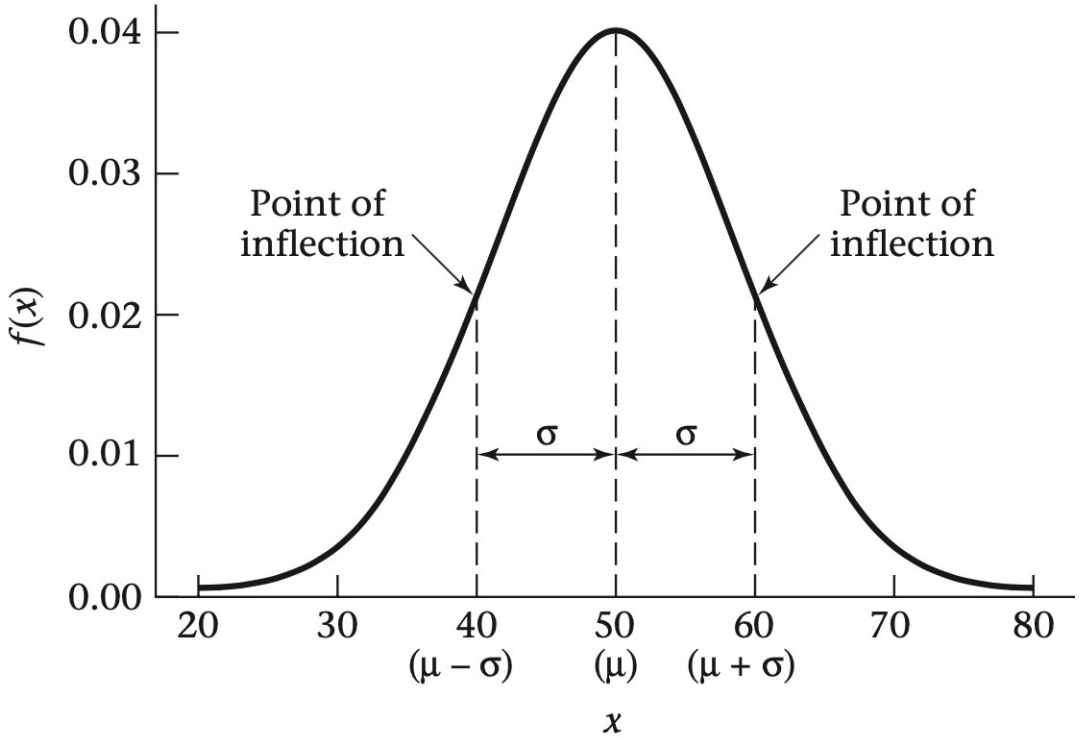
$$f(x) = \frac{1}{\sqrt{2\pi}\sigma} \exp\left[-\frac{1}{2\sigma^2}(x-\mu)^2 \right], \quad -\infty < x < \infty$$

for some parameters μ, σ, where σ > 0.

The **probability-density function** of the random variable *X* is a function such that the area under the density-function curve between any two points *a* and *b* is equal to the probability that the random variable *X* falls between *a* and *b*. Thus, the total area under the density-function curve over the entire range of possible values for the random variable is 1.

Normal/Gaussian Distribution (CPD)

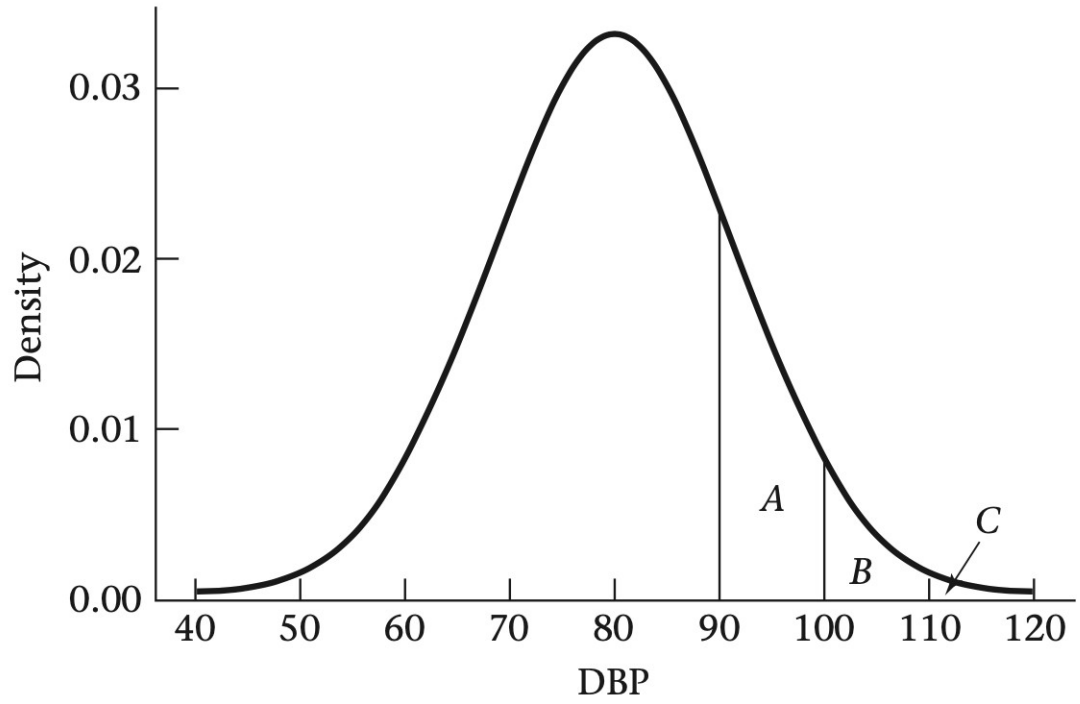
The pdf for a normal distribution with mean μ (50) and variance σ² (100)



- *A point of inflection is a point at which the slope of the curve changes direction. In the above Figure, the slope of the curve increases to the left of* μ − σ *and then starts to decrease to the right of* μ − σ *and continues to decrease until reaching* μ + σ, *after which it starts increasing again.*

Normal/Gaussian Distribution (CPD)

The pdf of DBP in 35- to 44-year-old men



Conditional Probability

- Probability of H on throwing a fair coin (all outcomes have equal probability).
- Probability of H,H,H,H,H on throwing a coin 5 times.
- Conditional Probability: $P(A|B)$
- Independent events: If and only if

$$P(A|B) = P(A)$$

Probability of getting a H (when we have already got T)

or equivalently,

$$P(A \cap B) = P(A)P(B)$$

P(H,T) P(H) P(T)

¼ ½ ½

Conditional Probability

defined

$$P(A|B) \triangleq \frac{P(A \cap B)}{P(B)}$$

A: 2 on dice

B: Even number on dice

$$\frac{P(2 \text{ \& } Even)}{P(Even)} \qquad \frac{1/6}{3/6} \qquad 1/3$$

$$A \subseteq B \Longrightarrow P(A) \leq P(B)$$

Bayes theorem

$$P(A \mid B) = \frac{P(B \mid A) \cdot P(A)}{P(B)}$$

A: 2 on dice

B: Even number on dice

A, B = events

$P(A|B)$ = probability of A given B is true

$P(B|A)$ = probability of B given A is true

$P(A), P(B)$ = the independent probabilities of A and B

$$\frac{P(Even \mid 2) \times P(2)}{P(Even)} \qquad \frac{1/6}{3/6} \qquad 1/3$$

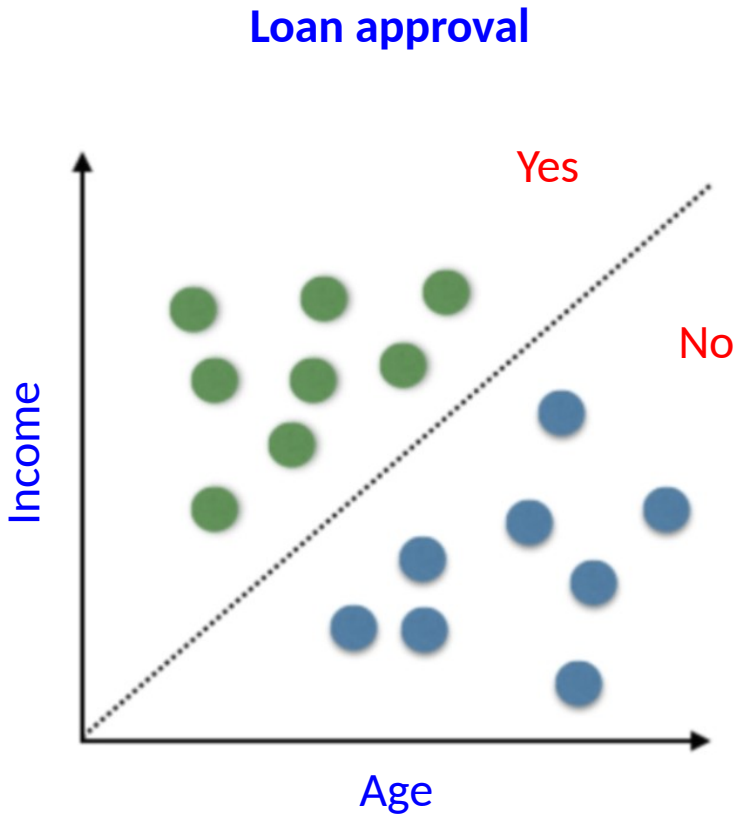
Machine Learning

(Classification, Regression)

Deepak Sharma
BSBE and DS&AI
IIT Roorkee

Linear classification

Linear classification refers to categorizing a set of data points to a discrete class based on a linear combination of its variables.



Linear classification

$y = mx + c$

$y - mx - c = 0$

$w_2x_2 + w_1x_1 + w_0x_0 = 0$ (added coordinate $x_0 = 1$ for w_0)

$$\sum_{i=0}^{d} w_i x_i \quad 0$$

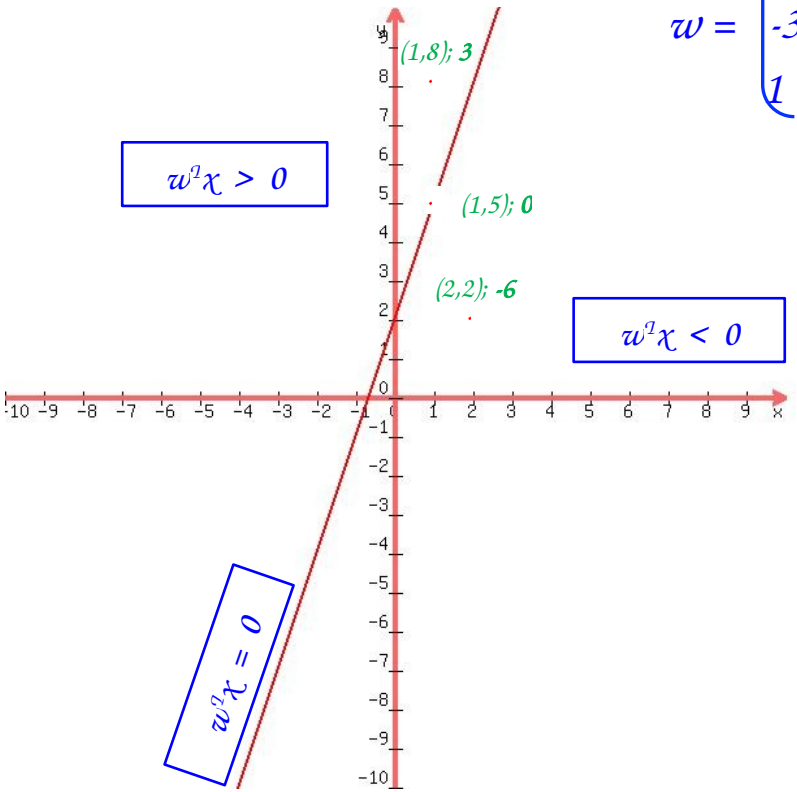
$w^T x = 0$

$y = 3x + 2$

$x_2 = 3x_1 + 2$

$x_2 - 3x_1 - 2 = 0$

$$w = \begin{pmatrix} 2\ (w_0) \\ -3\ (w_1) \\ 1\ (w_2) \end{pmatrix} \qquad x = \begin{pmatrix} 1\ (x_0) \\ 1\ (x_1) \\ 5\ (x_2) \end{pmatrix}$$

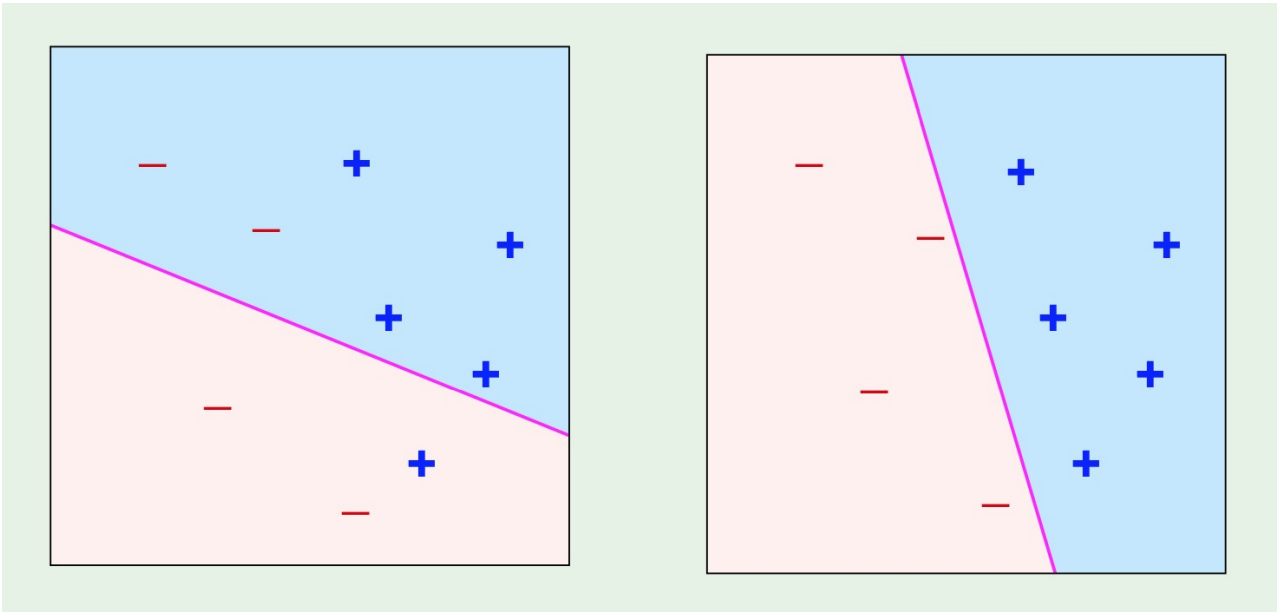


Linear classification

Binary classification [(0,1); (-1,1); (T,F); (Y,N)]:

$$h(\mathbf{x}) = \text{sign}(\mathbf{w}^{\mathrm{T}}\mathbf{x})$$

-ve: -1 (False)
+ve: 1 (True)



Misclassified data

Perfectly classified data

Perceptron Learning Algorithm (PLA)

$t = 0$ # t (iteration) = 0, 1, 2, …

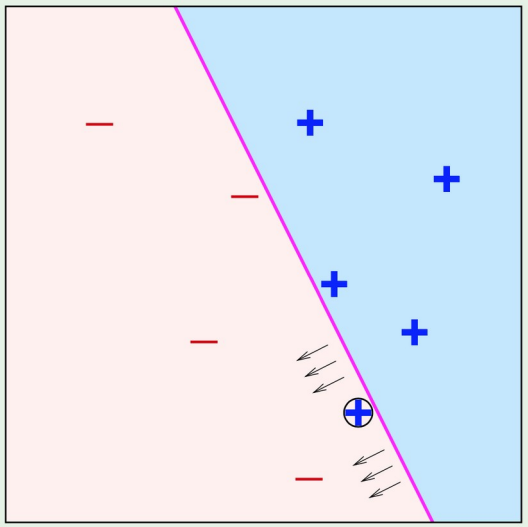
w(t) = 0 # vector

while $y(t) \neq \text{sign}(\mathbf{w}^{\mathrm{T}}(t)\mathbf{x}(t))$ # a point [x(t), y(t)] is misclassified

$$\mathbf{w}(t+1) = \mathbf{w}(t) + y(t)\mathbf{x}(t)$$ # update rule

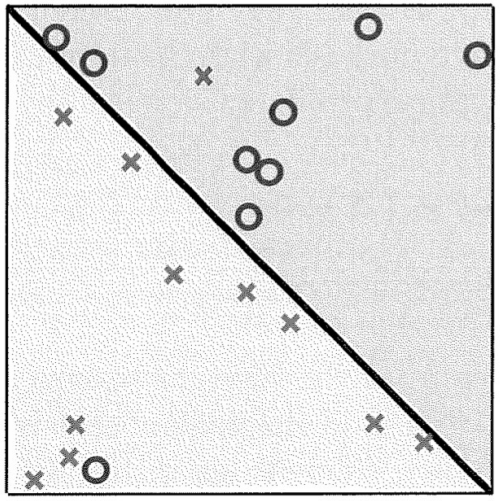
return w(t)

In program, randomly pick misclassified point



Pocket algorithm

- *The algorithm keeps 'in its pocket' the best weight vector encountered up to iteration* t.
- *At the end, the best weight vector will be reported as the final hypothesis.*

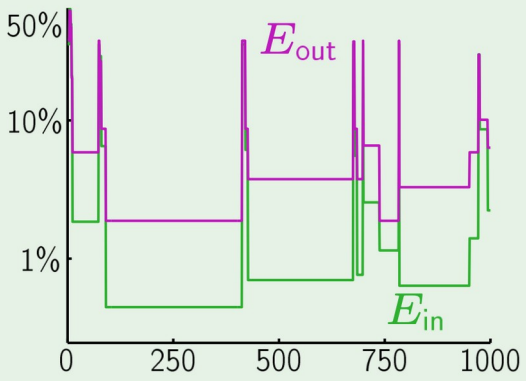


The pocket algorithm:

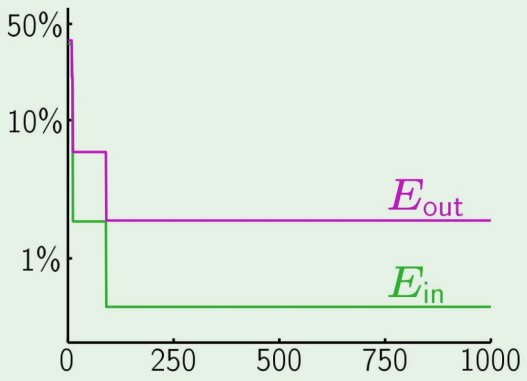
1: Set the pocket weight vector $\hat{\mathbf{w}}$ to $\mathbf{w}(0)$ of PLA.
2: **for** $t = 0, \ldots, T-1$ **do**
3: Run PLA for one update to obtain $\mathbf{w}(t+1)$.
4: Evaluate $E_{\text{in}}(\mathbf{w}(t+1))$.
5: If $\mathbf{w}(t+1)$ is better than $\hat{\mathbf{w}}$ in terms of E_{in}, set $\hat{\mathbf{w}}$ to $\mathbf{w}(t+1)$.
6: Return $\hat{\mathbf{w}}$.

PLA vs. Pocket

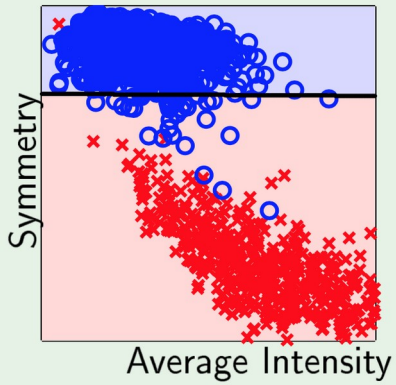
PLA:



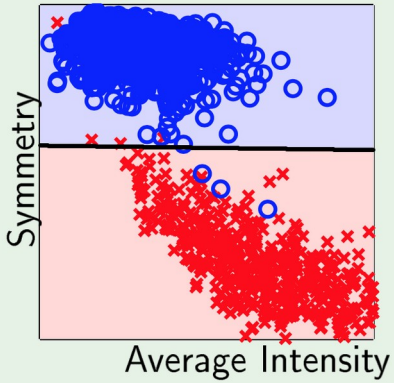
Pocket:



PLA:

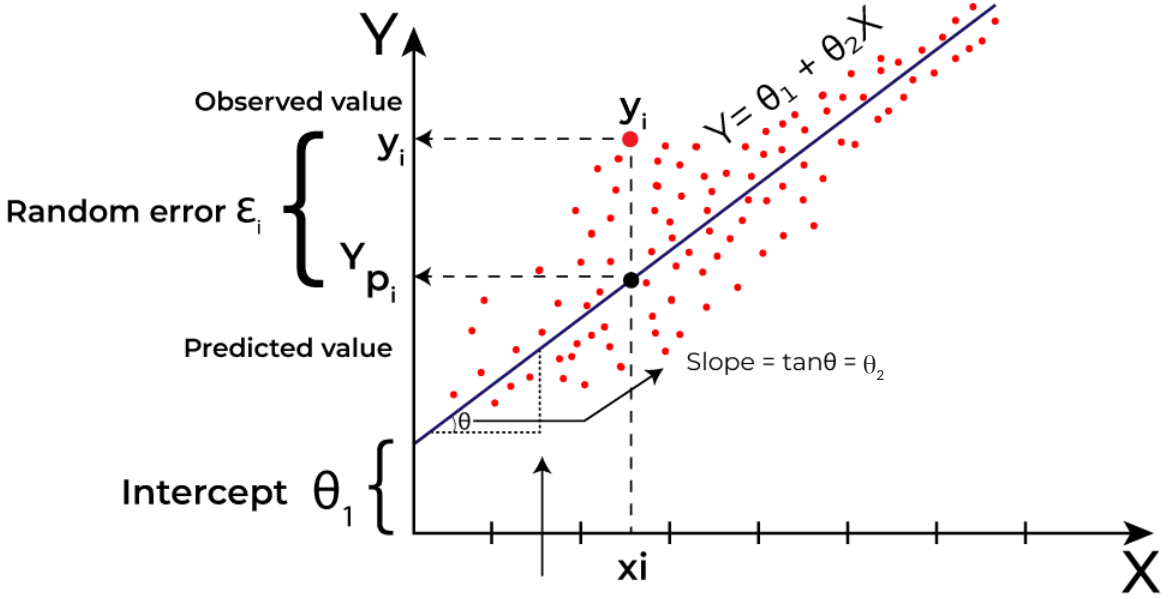


Pocket:



Linear Regression

Best fit line

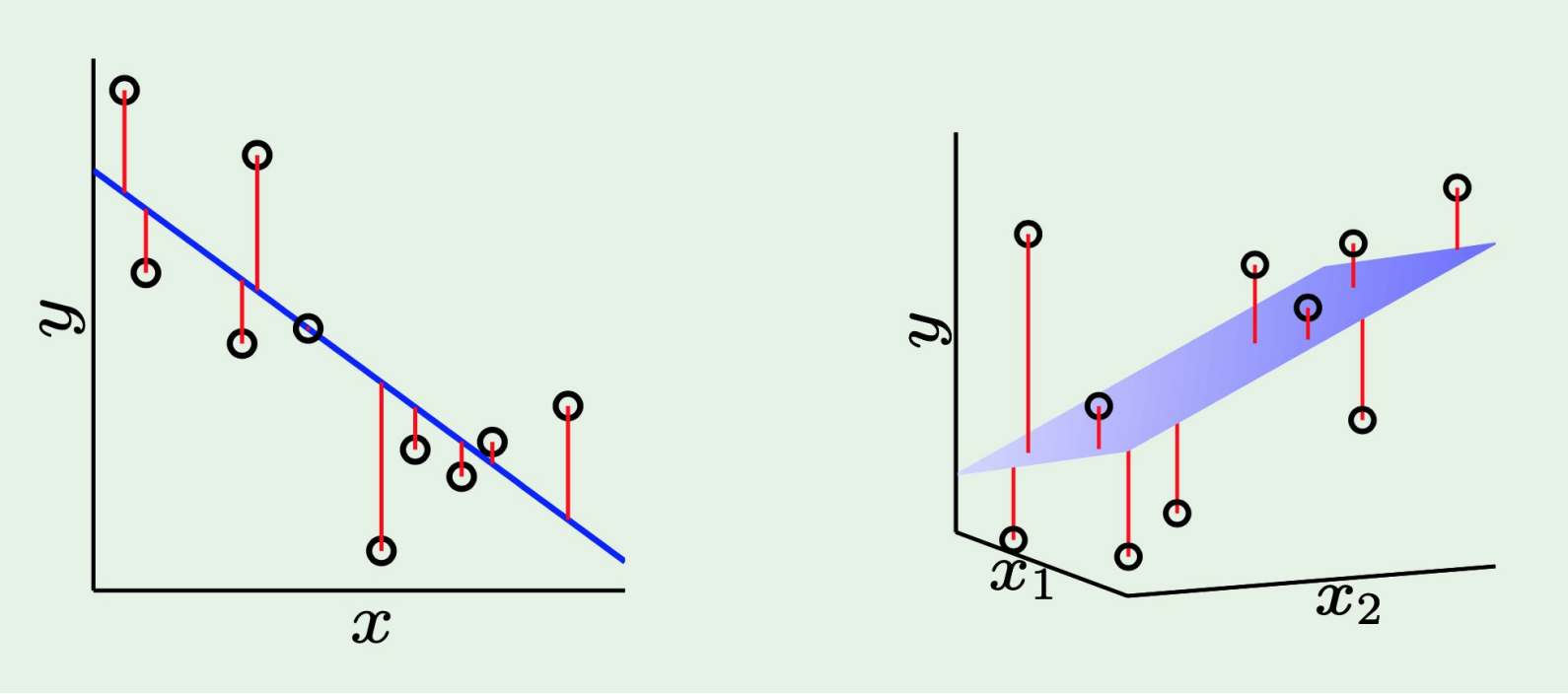


Error = $\frac{1}{n}\sum_{i=1}^{n}(\hat{y}_i - y_i)^2$ MSE (Mean square error); $\hat{y}_i$ Predicted value

in-sample error: $E_{\text{in}}(h) = \frac{1}{N}\sum_{n=1}^{N}(h(\mathbf{x}_n) - y_n)^2$

Best fit line: minimize error

Linear Regression



One dimension (line) *Two dimensions (hyperplane)*

The sum of squared errors is minimized

Linear Regression

$$E_{\text{in}}(\mathbf{w}) = \frac{1}{N}\sum_{n=1}^{N}(\mathbf{w}^{\mathsf{T}}\mathbf{x}_n - y_n)^2$$

$$= \frac{1}{N}\|\mathbf{X}\mathbf{w} - \mathbf{y}\|^2$$

L_2-norm

$$x^T y = y^T x$$

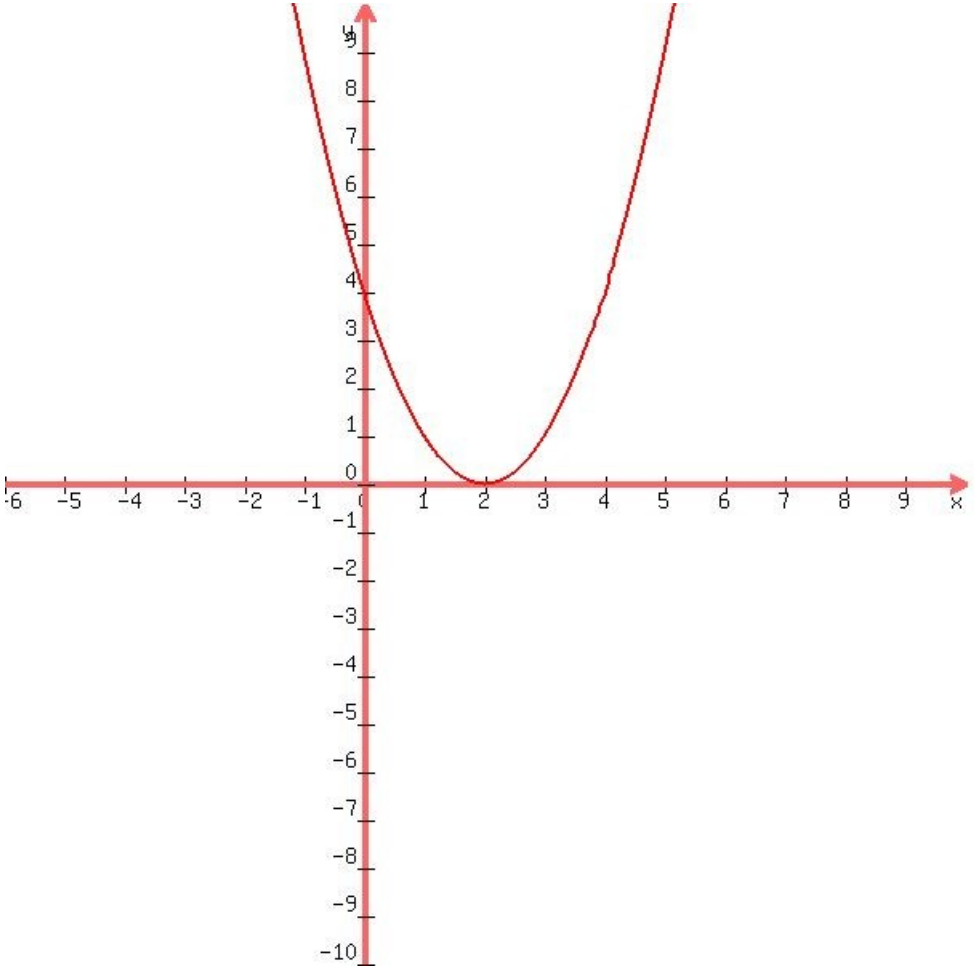
where $\mathbf{X} = \begin{bmatrix} —\mathbf{x}_1{}^{\mathsf{T}}— \\ —\mathbf{x}_2{}^{\mathsf{T}}— \\ \vdots \\ —\mathbf{x}_N{}^{\mathsf{T}}— \end{bmatrix}$, $\mathbf{y} = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_N \end{bmatrix}$

$\mathcal{N} \times n$ $\quad$ $n \times 1$ $\quad$ $\mathcal{N} \times 1$

X $\quad$ w $\quad$ Xw

n would actually be (n+1) since we add x_0

Minima



$f(x) = (x-2)^2$

$f(x) = x^2-4x+4$

$f'(x) = 2x - 4$

$f'(x) = 2x-4 = 0$

$x = 2$ (Minima)

$\min_x f(x) = 0$

$\operatorname{argmin}_x f(x) = 2$

Linear Regression

$$\mathbf{w}_{\text{lin}} = \underset{\mathbf{w}\in\mathbb{R}^{d+1}}{\operatorname{argmin}} E_{\text{in}}(\mathbf{w})$$

Minimizing E_{in}

$$E_{\text{in}}(\mathbf{w}) = \frac{1}{N}\|\mathrm{X}\mathbf{w} - \mathbf{y}\|^2$$

$$\nabla E_{\text{in}}(\mathbf{w}) = \frac{2}{N}\mathrm{X}^{\mathsf{T}}(\mathrm{X}\mathbf{w} - \mathbf{y}) = \mathbf{0}$$

$$\mathrm{X}^{\mathsf{T}}\mathrm{X}\mathbf{w} = \mathrm{X}^{\mathsf{T}}\mathbf{y}$$

$$\mathbf{w} = \mathrm{X}^{\dagger}\mathbf{y} \quad \text{where} \quad \mathrm{X}^{\dagger} = (\mathrm{X}^{\mathsf{T}}\mathrm{X})^{-1}\mathrm{X}^{\mathsf{T}}$$

$\mathrm{X}^{\dagger}$ is the '**pseudo-inverse**' of X

$(X^TX)^{-1}X^TX$ = *Identity matrix*

Linear Regression algorithm

Loan prediction

$\mathbf{x}_i$ Vector with input features (Age, Income etc)

y_i Credit limit

1: Construct the matrix $\mathbf{X}$ and the vector $\mathbf{y}$ from the data set $(\mathbf{x}_1, y_1), \cdots, (\mathbf{x}_N, y_N)$ as follows

$$\underbrace{\mathbf{X} = \begin{bmatrix} —\mathbf{x}_1^\top— \\ —\mathbf{x}_2^\top— \\ \vdots \\ —\mathbf{x}_N^\top— \end{bmatrix}}_{\text{input data matrix}}, \qquad \underbrace{\mathbf{y} = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_N \end{bmatrix}}_{\text{target vector}}.$$

2: Compute the pseudo-inverse $\mathbf{X}^\dagger = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top$.

3: Return $\mathbf{w} = \mathbf{X}^\dagger \mathbf{y}$.

Gradient descent

It is a technique for minimizing a twice-differentiable function.

$f(x) = x^2 - 5x + 6$

$f'(x) = 2x - 5$

$x_0 = -2$

$x_1 = x_0 - f'(x) \big|_{x_0}$

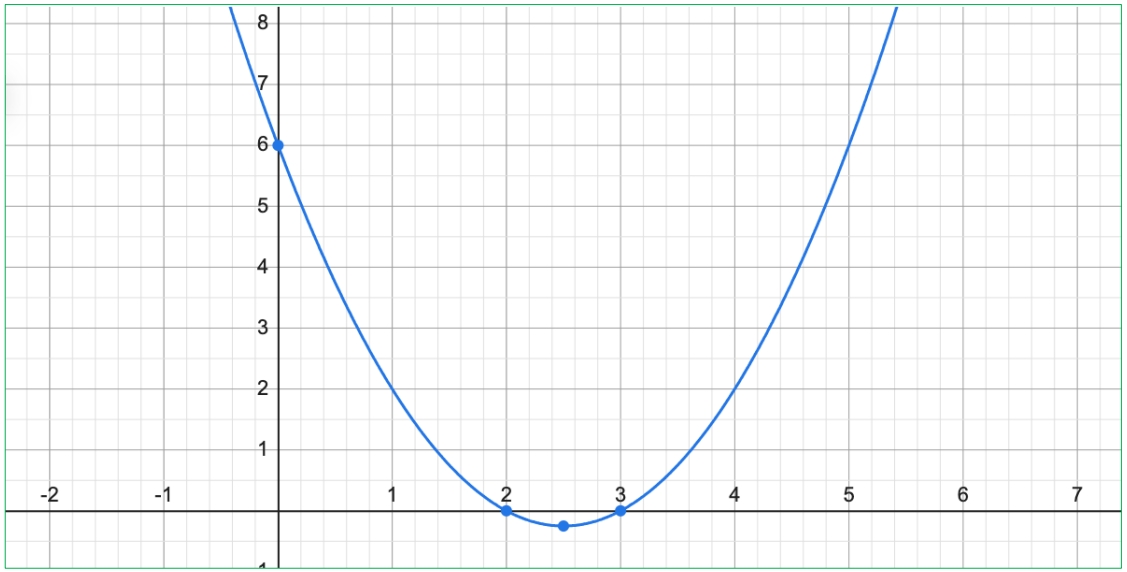
$= -2 - (-9)$

$= 7$

$x_2 = x_1 - f'(x) \big|_{x_1}$

$= 7 - 9$

$= -2$



$x_{(t+1)} = x_t - \eta f'(x) \big|_{x_t}$

$\eta = 1$

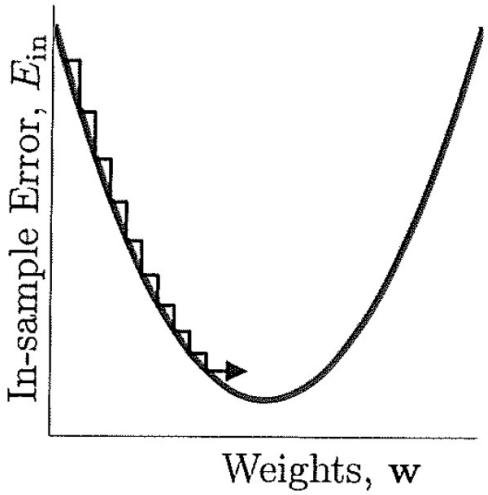
$\eta = 0.1$

Gradient descent

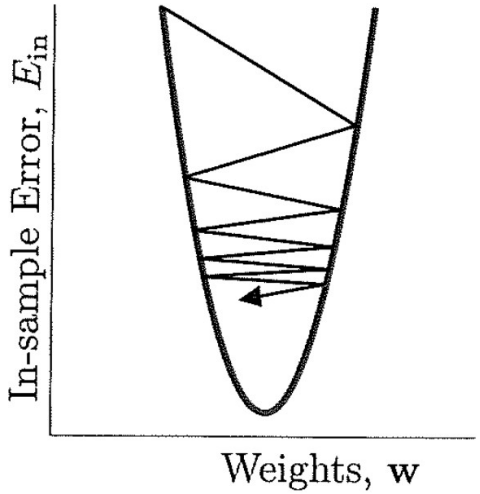
$x_{(t+1)} = x_t - \eta f'(x) \Big|_{x_t}$

η *Hyperparameter (Optimization)*

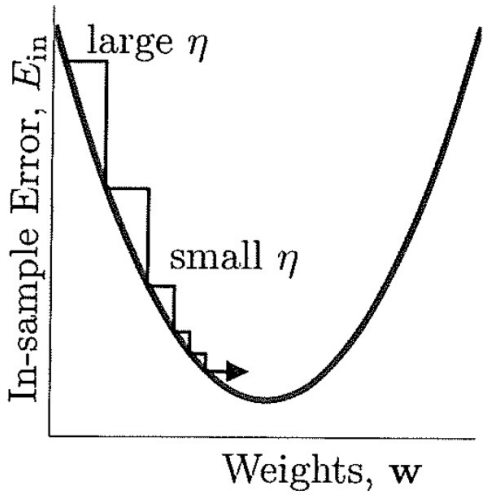
external parameter [not part of function f(x)] used to train ML model



η too small



η too large



variable η – just right

Adaptive η

η Learning rate

Eg. Adam optimizer

$$x_{(t+1)} = x_t - \eta \frac{f'(x)}{f''(x)} \Big|_{x_t}$$

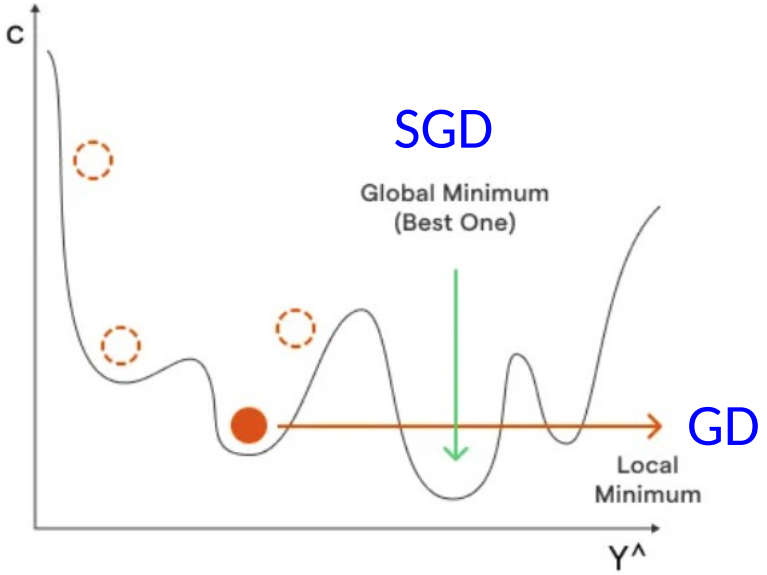
Stochastic gradient descent

- In SGD, instead of using the entire dataset for each iteration, only a single random training example (or a small batch) is selected to calculate the gradient and update the model parameters.

- This random selection introduces randomness into the optimization process, hence the term 'stochastic" in stochastic gradient descent.

- The randomness allows the model to escape from local minima and reach global minima.

- It is also faster.



Logistic Regression

$$f(x) = \frac{1}{1 + e^{-x}}$$

- The target variable can take only discrete values for a given set of features.
- The model builds a regression model to predict the probability of a given data entry.
- Similar to linear regression, logistic regression uses a linear function and, in addition, makes use of the 'sigmoid' function.
- Logistic regression can be further classified into three categories:

  (i) Binomial: target variable assumes only two values since binary. Eg: '0' or '1'.

  (ii) Multinomial: target variable assumes ≥ 3 unordered values since multinomial. Eg: 'Class A,' 'Class B,' and 'Class C'.

  (iii) Ordinal: target variable assumes ordered values since ordinal. Eg: 'Very Good', 'Good', 'Average, 'Poor', 'Very poor'.

Logistic Regression

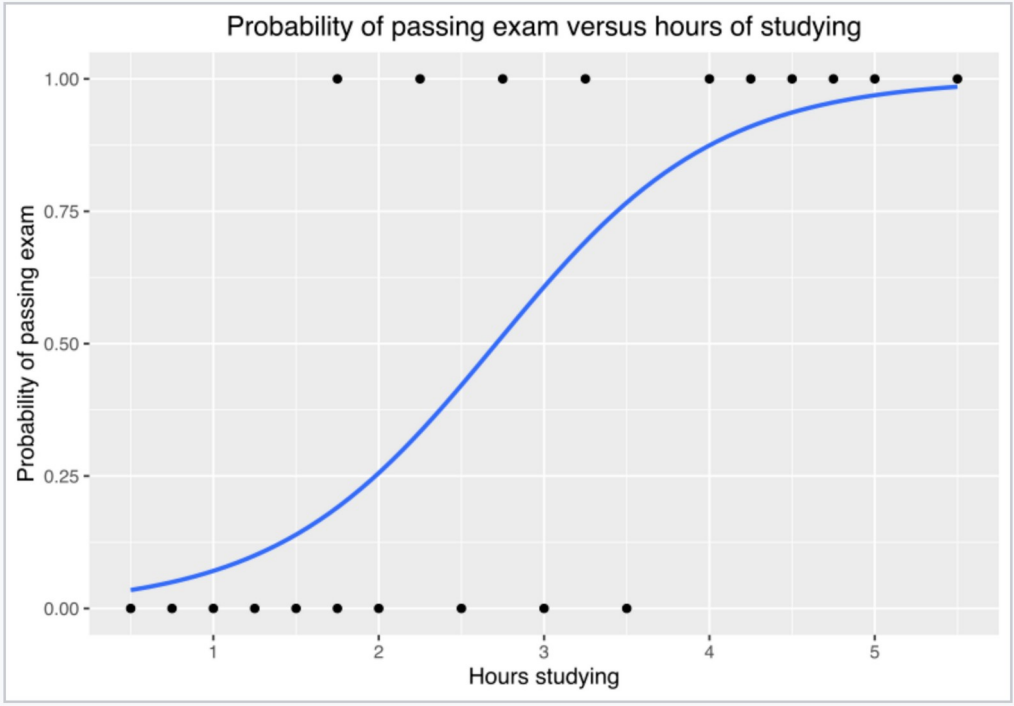
A group of 20 students spends between 0 and 6 hours studying for an exam. How does the number of hours spent studying affect the probability of the student passing the exam?

Hours (x_k)	0.50	0.75	1.00	1.25	1.50	1.75	2.00	2.25	2.50	2.75	3.00	3.25	3.50	4.00	4.25	4.50	4.75	5.00	5.50
Pass (y_k)	0	0	0	0	0	0	0	1	0	1	0	1	0	1	1	1	1	1	1

$$p(x) = \frac{1}{1 + e^{-(x-\mu)/s}}$$

μ is a location parameter (the midpoint of the curve, where $p(\mu)=1/2$)

s is a scale parameter (the larger the scale parameter, the more spread out the distribution)



Naïve Bayes classifier

$$P(A \mid B) = \frac{P(B \mid A) \cdot P(A)}{P(B)}$$

A, B	= events
$P(A\|B)$	= probability of A given B is true
$P(B\|A)$	= probability of B given A is true
$P(A), P(B)$	= the independent probabilities of A and B

- *It also lies in the domain of supervised ML and is based on two essential assumptions:-*

  (i) Conditional Independence: All features are independent of each other. This implies that one feature does not affect the performance of the other. This is the sole reason behind the 'Naive' in 'Naive Bayes'.

  (ii) Feature Importance: All features are equally important. It is essential to know all the features to make good predictions and get the most accurate results.

- *Naive Bayes is classified into three main types: Multinomial Naive Bayes, Bernoulli Naive Bayes, and Gaussian Bayes.*

Support vector machine (SVM)

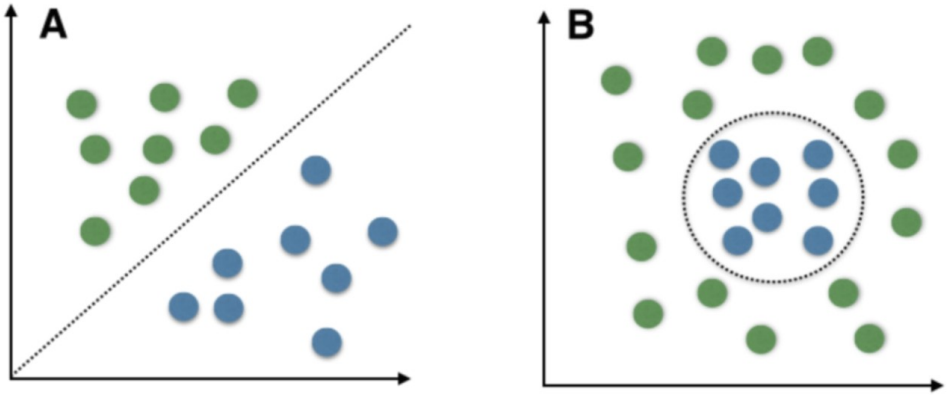
- It is a supervised ML algorithm used for regression/classification.
- It finds a hyper-plane that creates a boundary between various data types.
- The data points closest to the hyperplane that define the margin are called support vectors.
- It can be used for binary as well as multinomial classification problems. A binary classifier can be created for each class to perform multi-class classification.
- In case of SVM, the classifier with highest score is chosen as the output of SVM.

Support vector machine (SVM)

- SVM works very well with linearly separable data but can work for non-linearly separable data as well.
- SVMs use a mathematical trick called the **kernel trick** to map data into higher-dimensional spaces, making it easier to find a separating boundary.

Linear vs. Nonlinear SVMs:

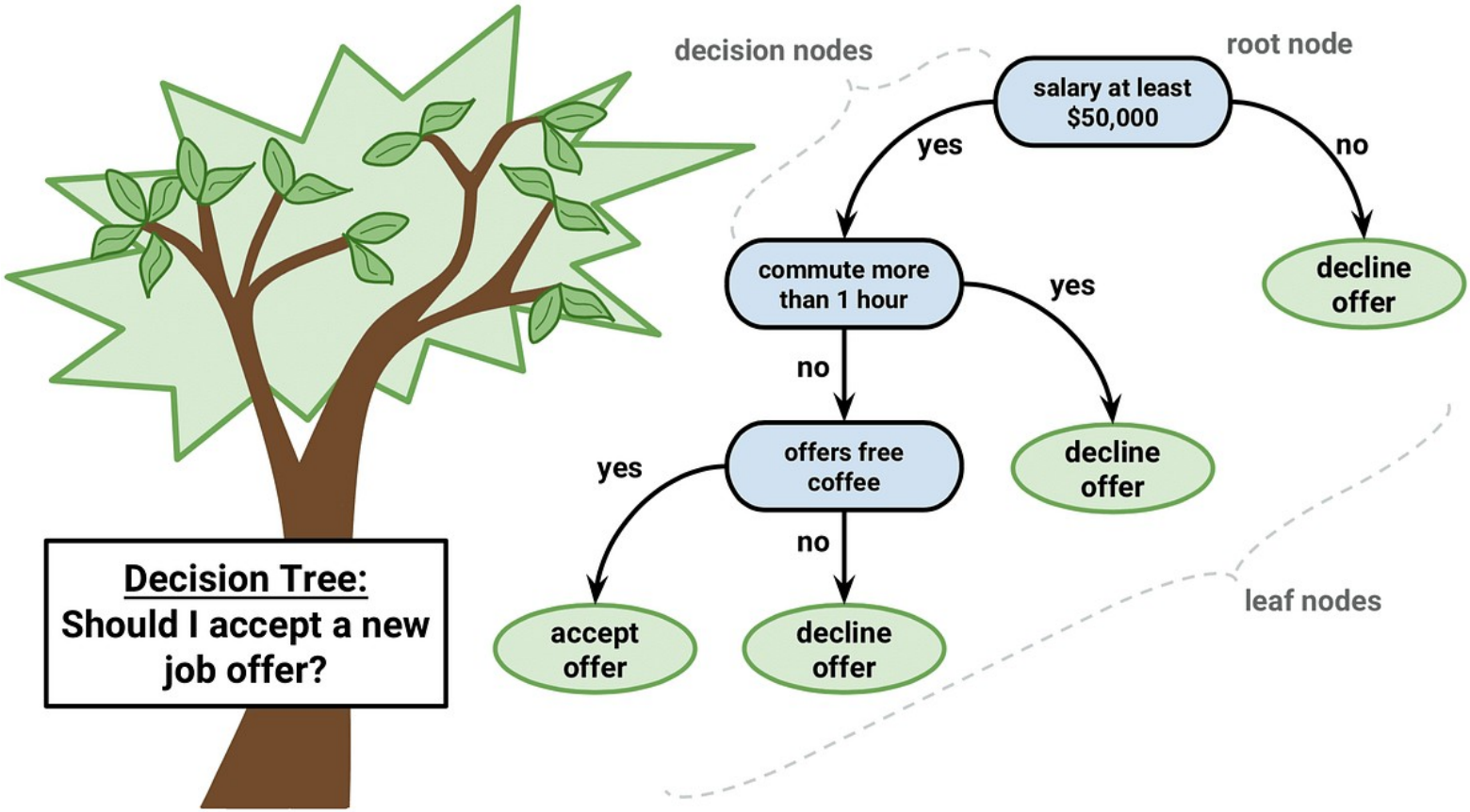
- **Linear SVMs:** Use a linear hyperplane for classification, suitable for linearly separable data.
- **Nonlinear SVMs:** Use kernel functions to map data into higher dimensions, allowing for more complex decision boundaries.



A: Linearly separable data B: Non-linearly separable data

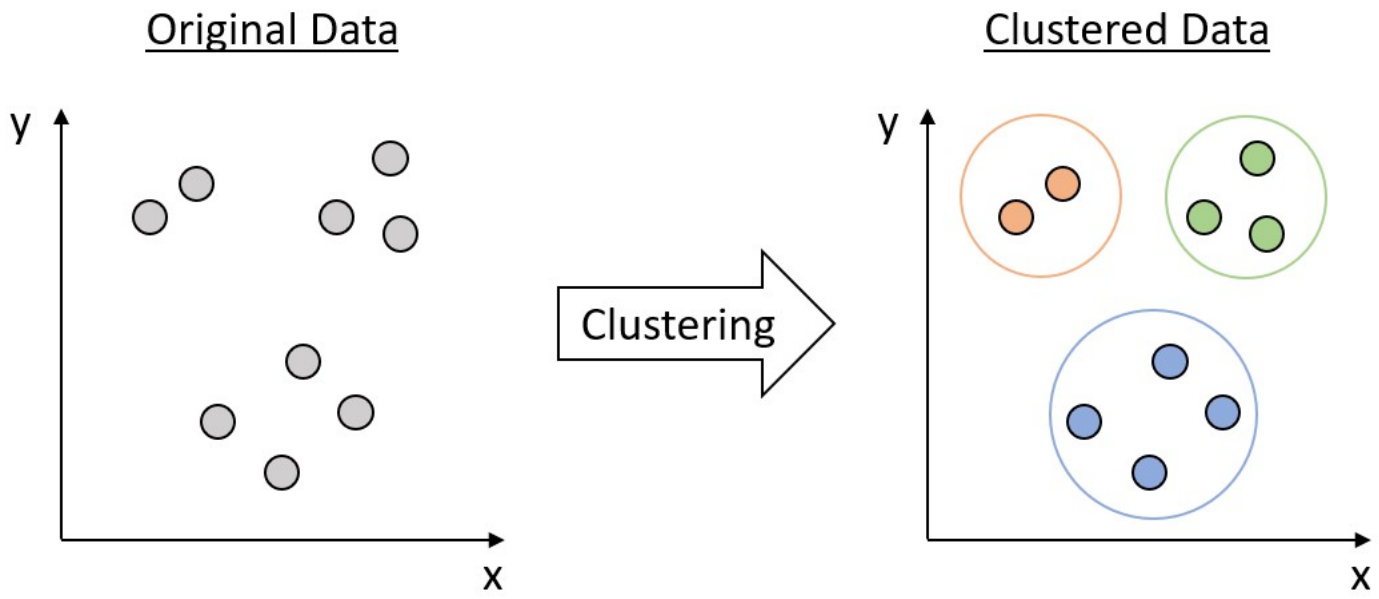
Decision Tree

- *It is a decision support recursive partitioning structure that uses a tree-like model of decisions and their possible consequences.*



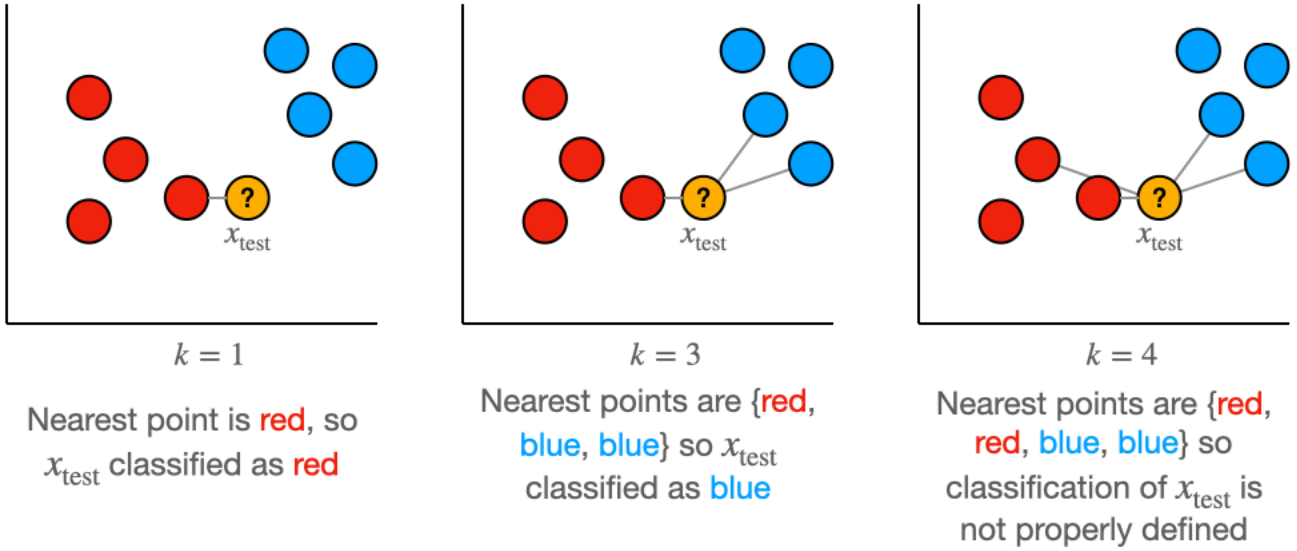
Clustering

- *Clustering is an unsupervised machine learning technique designed to group unlabeled examples based on their similarity to each other.*



kNN algorithm

- *k-nearest neighbors (kNN) algorithm is a non-parametric, supervised learning classifier, which uses proximity to make classifications or predictions about the grouping of an individual data point.*



- *In the case of a binary classifier (when we only have 2 classes to predict), k must be odd to avoid having undefined points.*

Machine Learning

(Evaluation metrics)

Deepak Sharma
BSBE and DS&AI
IIT Roorkee

Training, Validation & Testing datasets

- Training, testing, and validation datasets are used in the process of developing and evaluating a machine learning model.

- Training dataset: This dataset is used to train the machine learning model.

  - The model is trained by fitting the model to the training data. The model learns the patterns in the data and uses them to make predictions.

Training, Validation & Testing datasets

- *Validation dataset:* This dataset is used to tune the hyperparameters of the machine learning model.

  - The model is trained on the training data and then evaluated on the validation data.

  - The hyperparameters are adjusted based on the performance of the model on the validation data.

  - This helps to prevent overfitting of the model to the training data.

Training, Validation & Testing datasets

- *Testing dataset: This dataset is used to evaluate the performance of the machine learning model after it has been trained.*

  - The model is presented with new, unseen data and it makes predictions based on what it has learned from the training data.

  - The accuracy of these predictions is then used to evaluate the performance of the model.

Overfitting and Underfitting

- Overfitting and underfitting are two common issues faced while training machine learning models.

- Overfitting occurs when a model is trained too well on the training data and fits the noise in the data instead of the underlying pattern.

  - As a result, it performs well on the training data but poorly on the unseen data or validation data.

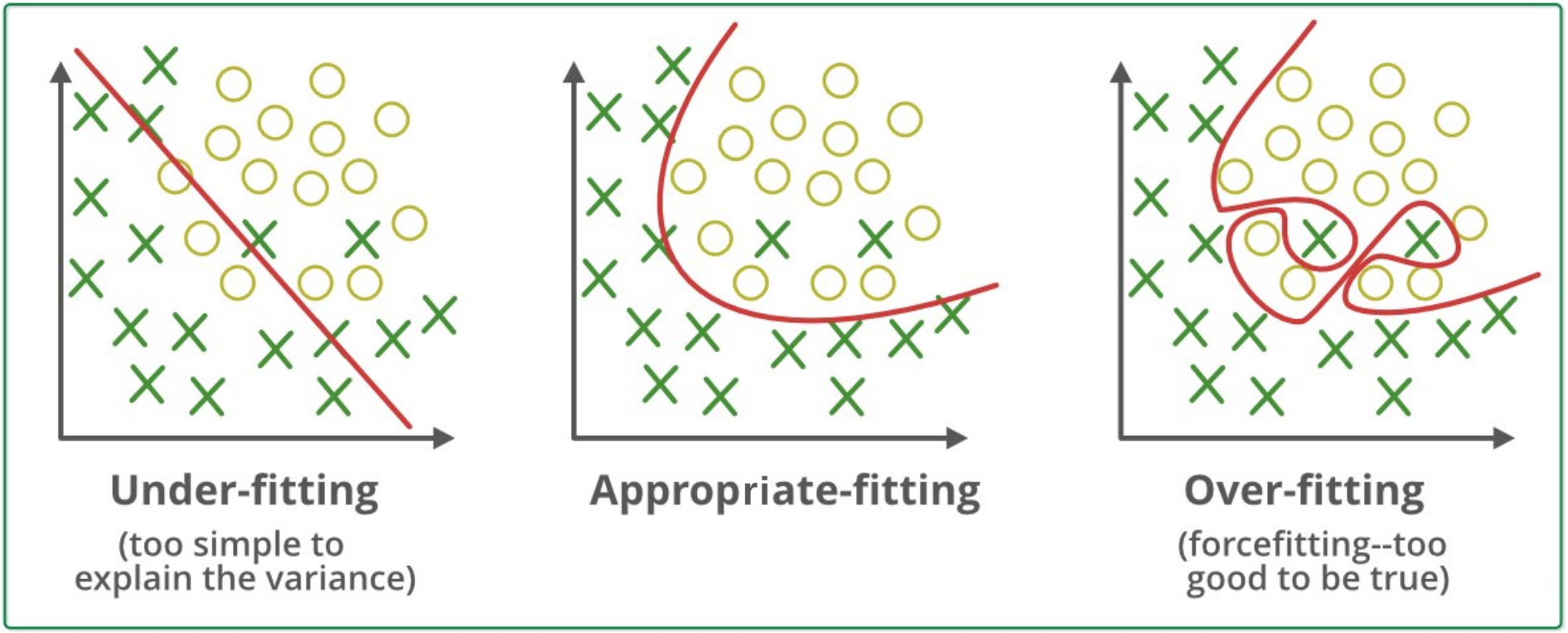
  - Overfitting can be identified by having a high accuracy on the training data but a low accuracy on the validation data.

Overfitting and Underfitting

- *Underfitting, on the other hand, occurs when a model is not complex enough to capture the underlying pattern in the data. It results in a low accuracy on both the training and validation data.*

- *It is important to strike a balance between overfitting and underfitting to build an effective model.*

Overfitting and Underfitting



Overfitting and Underfitting

A	B	C
Not interested in learning	Memorizing the lessons	Conceptual Learning
Class test ~50% Test ~47%	Class test ~98% Test ~69%	Class test ~92% Test ~89%
Under-fit/ biased learning	Over-fit/ Memorizing	Best-fit

Bias and Variance

- Bias and variance are two important concepts in machine learning that describe the error in a model's predictions.

- Bias refers to the error that is introduced by assuming that the relationship between the features and target is too simple.

  - A model with high bias pays little attention to the training data and oversimplifies the relationship between the features and target.

  - As a result, it often has a high training error and a high test error.

Bias and Variance

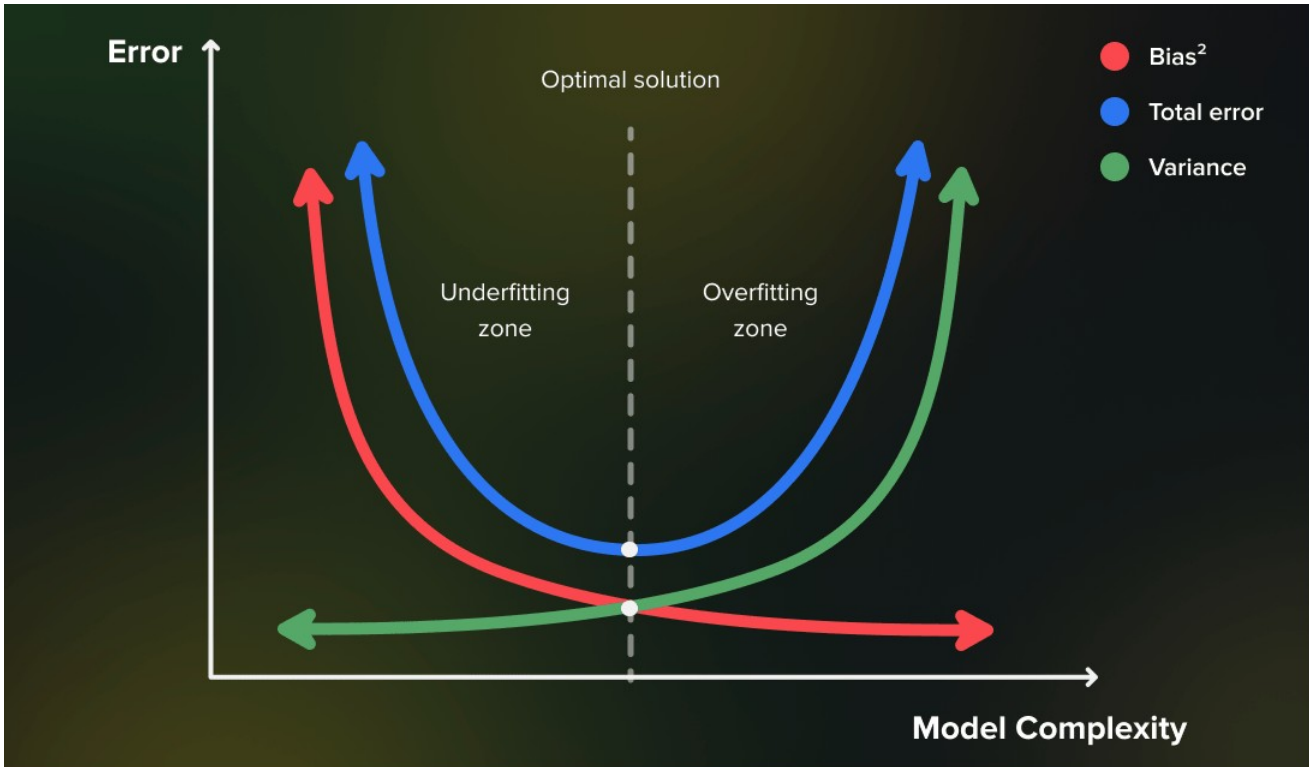
- *Variance, on the other hand, refers to the error that is introduced by the model being too complex and fitting the training data too closely.*

  - A model with high variance pays too much attention to the training data and overfits it, capturing the noise in the data as well as the underlying relationship.

  - As a result, it has a low training error but a high test error.

Bias and Variance

- *The goal in building a machine learning model is to find a balance between bias and variance to minimize the total error. This is often referred to as the bias-variance trade-off .*



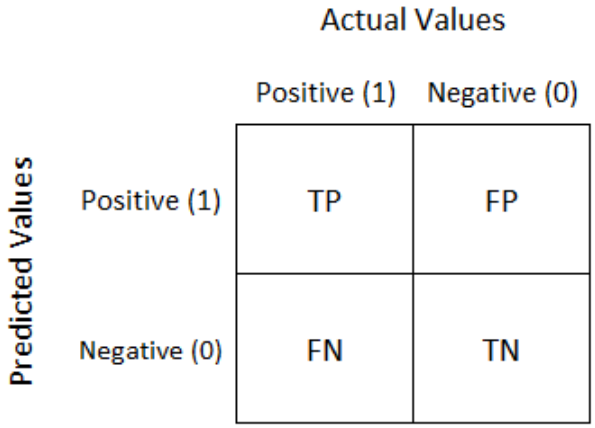
Model evaluation - Accuracy

- Ratio of correct predictions made by a classifier to the total number of predictions made.

$$\text{Accuracy} = \frac{\text{Number of correct predictions}}{\text{Total number of predictions}}$$

Model evaluation – Confusion matrix

- A 2-D table that shows the number of true positive, true negative, false positive, and false negative predictions made by the model.



- The entries in the matrix can then be used to calculate various performance metrics, such as precision, recall, F1-score, and AUC for the ROC curve.

Model evaluation - Precision

- The number of true positive predictions (i.e. positive predictions that are actually correct) divided by the total number of positive predictions made by the model.

$$Precision = \frac{TP}{TP + FP}$$

Model evaluation - Recall

- The proportion of actual positive instances that are correctly classified as positive by the model.
- Also called TPR (True Positive Rate) or Sensitivity.

$$Recall = \frac{TP}{TP + FN}$$

Model evaluation – F1 score

- A metric that combines precision and recall. It is calculated as the harmonic mean of precision and recall.

$$F1\text{-}score = \frac{2 * Precision * Recall}{Precision + Recall}$$

- The F1-score ranges between 0 and 1.

Model evaluation – ROC curve

- ROC (Receiver Operating Characteristic) curve is a graphical representation of the performance of a binary classification model as the discrimination threshold (probability threshold) is varied.
- It plots the true positive rate (TPR) against the false positive rate (FPR) at various threshold settings.
- The ROC curve is a useful tool for evaluating the trade-off between the true positive rate and the false positive rate of a classifier.

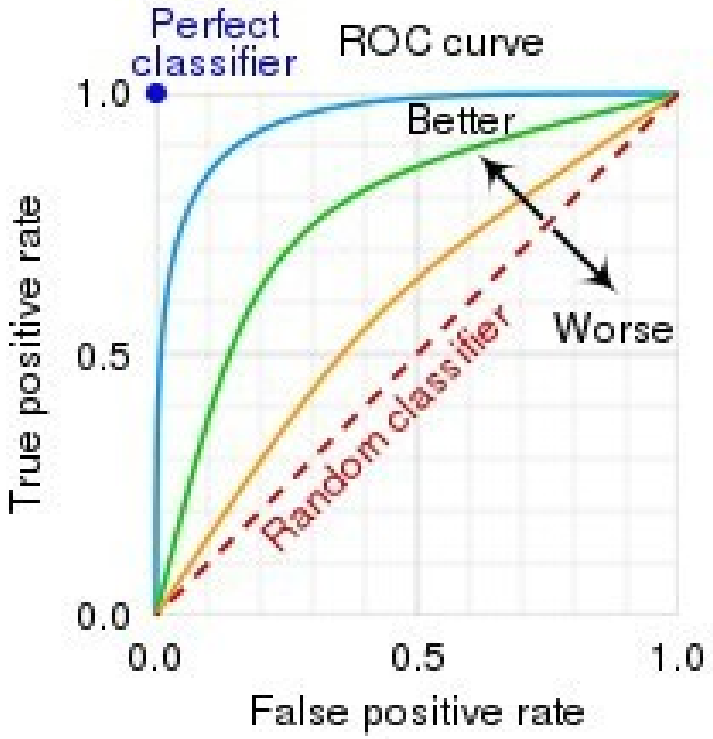
Model evaluation – ROC curve

- $TPR = TP / (TP + FN)$

  i.e. probability of detection

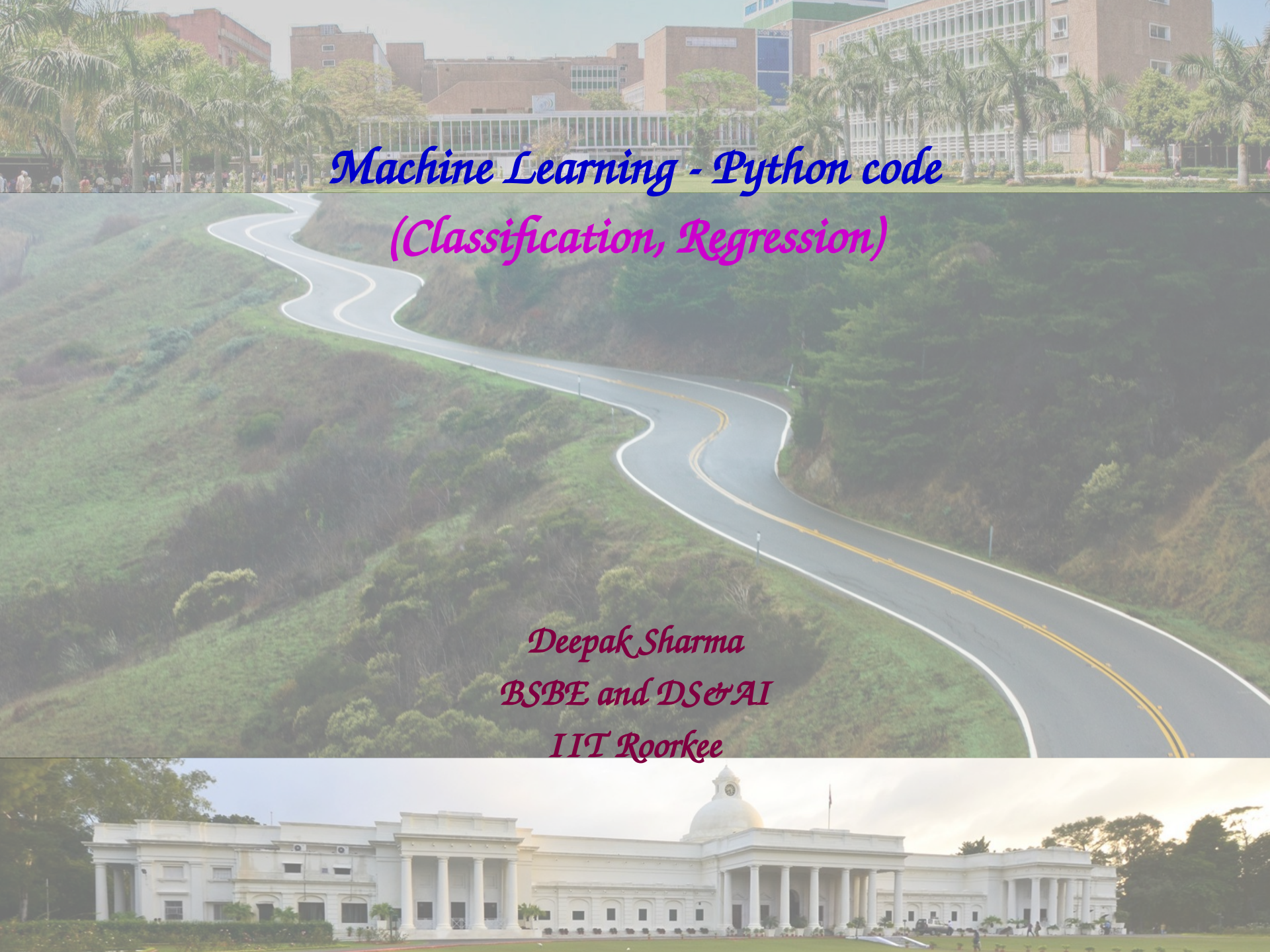
- $FPR = FP / (FP + TN)$

  i.e. probability of false alarm



Model evaluation – AUC

- The interpretation of the ROC curve is based on the Area Under the Curve (AUC), which summarizes the overall performance of the model.
- An AUC of 1 indicates a perfect model, while an AUC of 0.5 represents a random model.
- A higher AUC value indicates a better performance, with a larger area under the curve meaning a greater balance between TPR and FPR.



Machine Learning - Python code

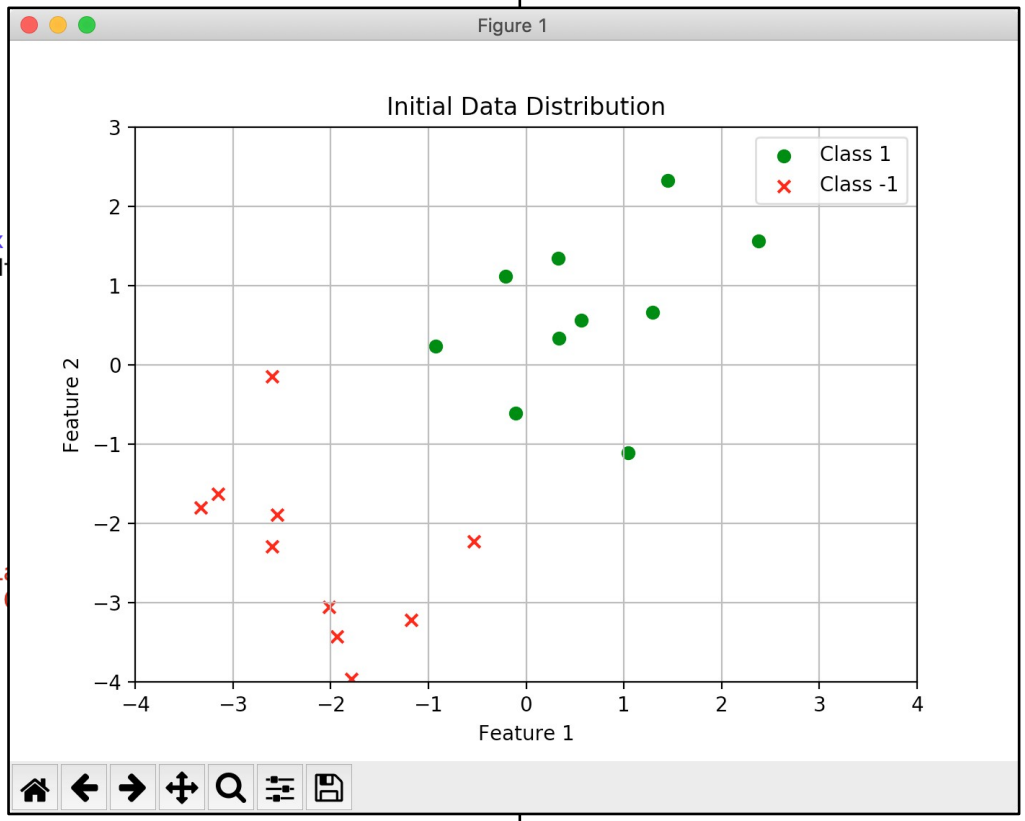
(Classification, Regression)

Deepak Sharma
BSBE and DS&AI
IIT Roorkee

Perceptron Learning Algorithm (PLA)

Lectures — vi PLA_demo_for_screenshots.py — 98×41

```
#!/Users/deepak/opt/anaconda3/bin/python3/

import numpy as np
import matplotlib.pyplot as plt
import warnings
warnings.filterwarnings("ignore")

# Step 1: Generate linearly separable data with two features and two classes
def generate_data(n):
    np.random.seed(42)
    # Class 1: centered around (0.8, 0.8)
    x1 = np.random.randn(n, 2) + [0.8, 0.8]
    # Class -1: centered around (-2, -2)
    x2 = np.random.randn(n, 2) + [-2, -2]

    X = np.vstack((x1, x2)) # size 2n x 2, 2n=N
    X = np.hstack((np.ones((X.shape[0],1)), X)) # size 2n x
    y = np.hstack((np.ones(n, dtype=int), -1 * np.ones(n, d
    return X, y

X, y = generate_data(n=10)
print("Number of samples in input data:", X.shape[0])
print("Number of features in input data:", X.shape[1])


# Step 2: Visualize data
def visualizeData(X, y):
    plt.figure(figsize=(7, 5))
    plt.scatter(X[y == 1][:, 1], X[y == 1][:, 2], label='Cl
    plt.scatter(X[y == -1][:, 1], X[y == -1][:, 2], label='
    plt.title("Initial Data Distribution")
    plt.xlabel("Feature 1")
    plt.ylabel("Feature 2")
    plt.xlim(-4, 4)
    plt.ylim(-4, 3)
    plt.legend()
    plt.grid()
    plt.show()

visualizeData(X, y)
```



Perceptron Learning Algorithm (PLA)

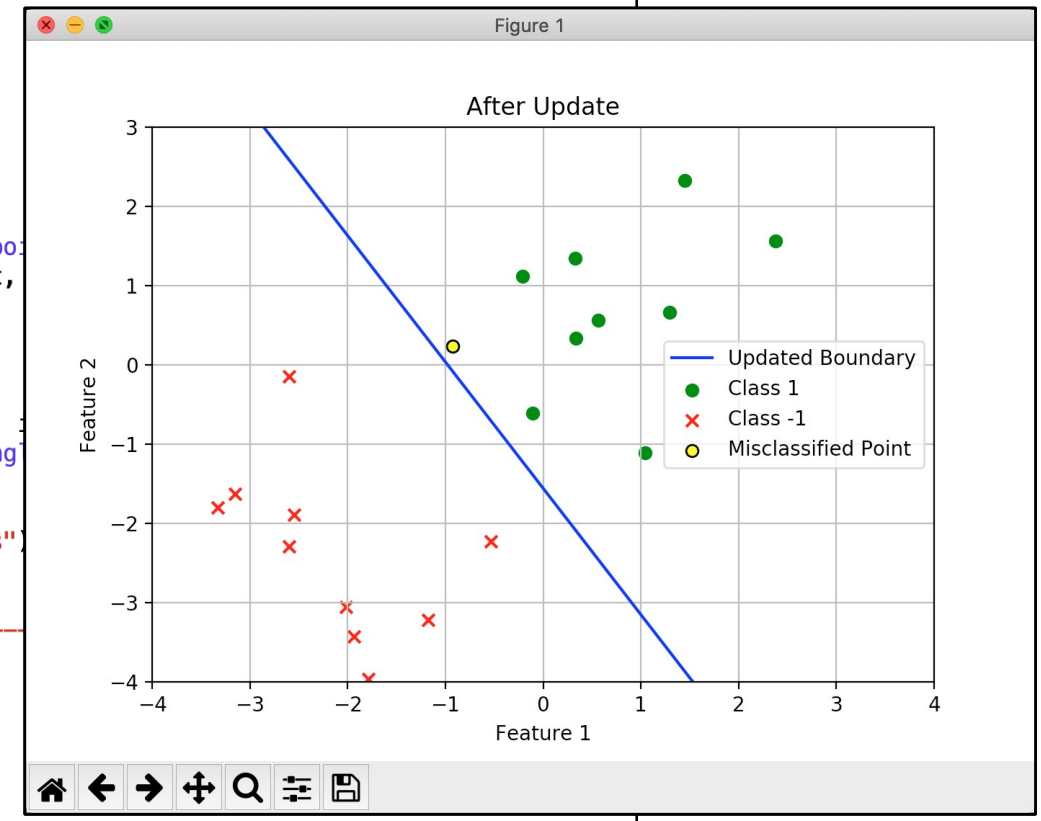
Lectures — vi PLA_demo_for_screenshots.py — 130×43

```
def plotBefore(X, y, w, misclassified_point, iteration):
    plt.figure(figsize=(7,5))
    # Plot current state
    plt.clf()
    plt.scatter(X[y == 1][:, 1], X[y == 1][:, 2], label='Class 1', marker='o', color='green')
    plt.scatter(X[y == -1][:, 1], X[y == -1][:, 2], label='Class -1', marker='x', color='red')

    # Decision boundary
    x_line = np.linspace(X[:, 1].min() - 1, X[:, 1].max() + 1, 10)
    if w[0]==0 and w[1]==0:
        y_line = 0*x_line
    else:
        y_line = -(w[0] + w[1] * x_line) / w[2]
    plt.plot(x_line, y_line, label=f'Iteration {iteration}', color='red')

    # Highlight misclassified point
    plt.scatter(misclassified_point[1], misclassified_point[2], color='yellow', label='Misclassified Point', edgecolor='black')

    plt.title("Before Update")
    plt.xlabel("Feature 1")
    plt.ylabel("Feature 2")
    plt.xlim(-4, 4)
    plt.ylim(-4, 3)
    plt.legend()
    plt.grid()
    plt.show()  # Pause to visualize

def plotAfter(X, y, w, misclassified_point, iteration):
    plt.figure(figsize=(7,5))
    # Plot updated decision boundary
    plt.clf()
    plt.scatter(X[y == 1][:, 1], X[y == 1][:, 2], label='Class 1', marker='o', color='green')
    plt.scatter(X[y == -1][:, 1], X[y == -1][:, 2], label='Class -1', marker='x', color='red')
    x_line = np.linspace(X[:, 1].min() - 1, X[:, 1].max() + 1, 10)
    y_line = -(w[0] + w[1] * x_line) / w[2]
    plt.plot(x_line, y_line, label='Updated Boundary', color='blue')
    plt.scatter(misclassified_point[1], misclassified_point[2], color='yellow', label='Misclassified Point', edgecolor='black')
    plt.title("After Update")
    plt.xlabel("Feature 1")
    plt.ylabel("Feature 2")
    plt.xlim(-4, 4)
    plt.ylim(-4, 3)
```

Perceptron Learning Algorithm (PLA)

Lectures — vi PLA_demo_for_screenshots.py — 100×36

```
def perceptron_algorithm(X, y, max_iterations=100):
    w = np.zeros(X.shape[1])  # Initialize weights

    for iteration in range(max_iterations):
        print(f"iteration: {iteration}")

        misclassified = False # flag

        for i in range(len(X)):
            if np.sign(np.dot(w, X[i])) != y[i]:
                misclassified = True # flag to true
                misclassified_point = X[i]

                # Plot current line and misclassified po
                plotBefore(X, y, w, misclassified_point,
                # Update weights
                w = w + y[i]*X[i]

                # Plot updated line
                plotAfter(X, y, w, misclassified_point,
                break  # Exit loop after updating a sing

        if not misclassified:
            print(f"Converged in {iteration} iterations"
            break
        print("Current weights:", w)
        print("----------------------------------------

    plt.show()
    return w

# Step 4: Run the algorithm and visualize progress
final_weights = perceptron_algorithm(X, y)
print("Final weights:", final_weights)
```

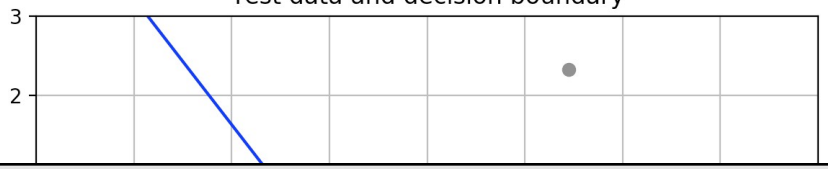


Perceptron Learning Algorithm (PLA)

Lectures — vi PLA_demo_for_screenshots.py — 100×29

```
#Prediction
XTest, yTest = generate_data(2)
XTest, yTest
#print(XTest, yTest)

plt.figure(figsize=(7,5))
# Plot decision boundary
plt.clf()
plt.scatter(XTest[:, 1], XTest[:, 2], label='Test data
x_line = np.linspace(X[:, 1].min() - 1, X[:, 1].max()
y_line = -(final_weights[0] + final_weights[1] * x_lin
plt.plot(x_line, y_line, label='Decision Boundary', co
plt.title("Test data and decision boundary")
plt.xlabel("Feature 1")
plt.ylabel("Feature 2")
plt.xlim(-4, 4)
plt.ylim(-4, 3)
plt.legend()
plt.grid()
plt.show()

predicted_labels = []
for i in range(XTest.shape[0]):
    predicted_labels.append(int(np.sign(np.dot(final_w

print(yTest)
print(predicted_labels)
```

Figure 1



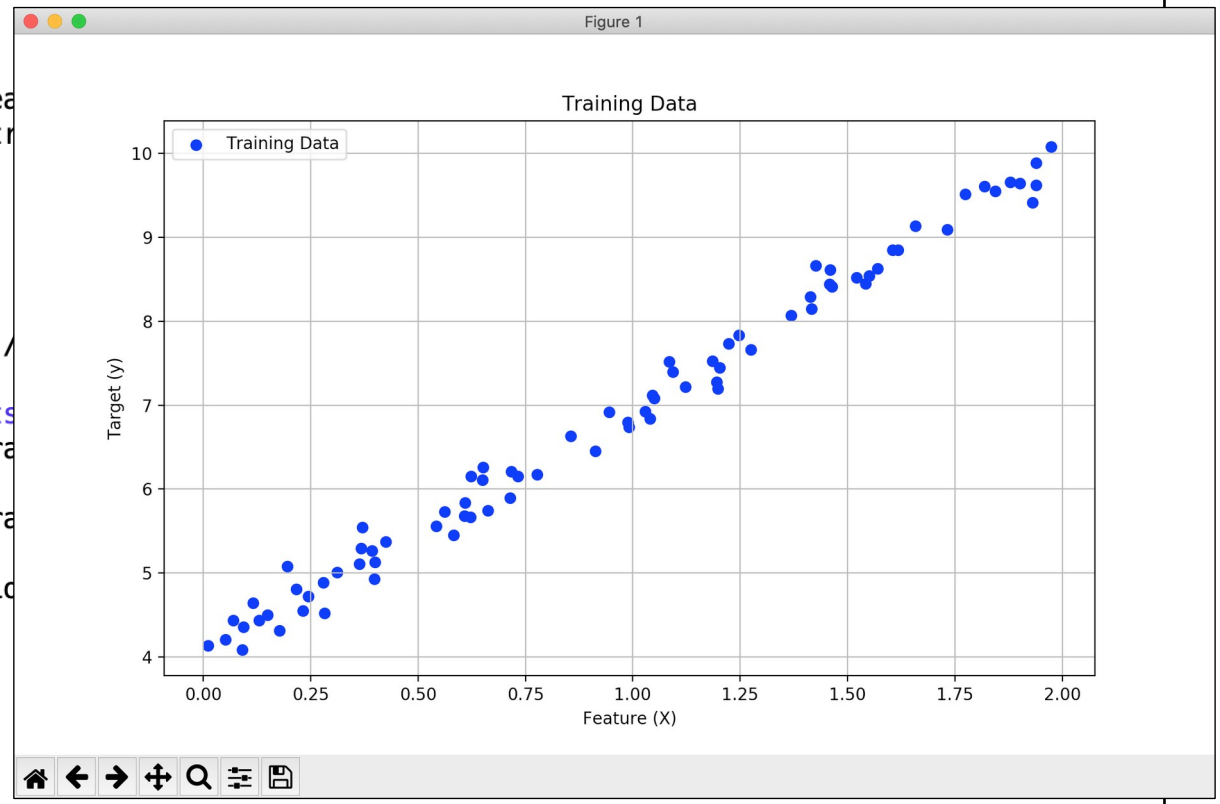
Jitin — -bash — 78×23

```
(base) Deepaks-MacBook-Pro:Jitin deepak$ ./PLA_demo.py
Number of samples in input data: 20
Number of features in input data: 3
iteration: 0
Current weights: [1.         1.29671415 0.6617357 ]
------------------------------------------------------------
iteration: 1
Current weights: [2.         0.37179632 0.89944817]
------------------------------------------------------------
iteration: 2
Current weights: [1.         2.97350293 1.04716999]
------------------------------------------------------------
iteration: 3
Current weights: [2.         2.0485851  1.28488246]
------------------------------------------------------------
iteration: 4
Converged in 4 iterations
Final weights: [2.         2.0485851  1.28488246]
[ 1  1 -1 -1]
[1, 1, -1, -1]
(base) Deepaks-MacBook-Pro:Jitin deepak$
```

Linear Regression

Lectures — vi LinearReg_demo_for_screenshots.py — 107×29

```
#!/Users/deepak/opt/anaconda3/bin/python3/

import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import Linea
from sklearn.model_selection import tr

# Generate synthetic data
np.random.seed(42)
n=100
X = 2 * np.random.rand(n, 1)
y = 4 + 3 * X + np.random.randn(n, 1)/

# Split into training and testing sets
X_train, X_test, y_train, y_test = tra

def plot_synthetic_data(X_train, y_tra
    plt.figure(figsize=(10, 6))
    plt.scatter(X_train, y_train, colo
    plt.xlabel('Feature (X)')
    plt.ylabel('Target (y)')
    plt.title('Training Data')
    plt.legend()
    plt.grid(True)
    plt.show()
# Call the function to visualize the data
plot_synthetic_data(X_train, y_train)
```

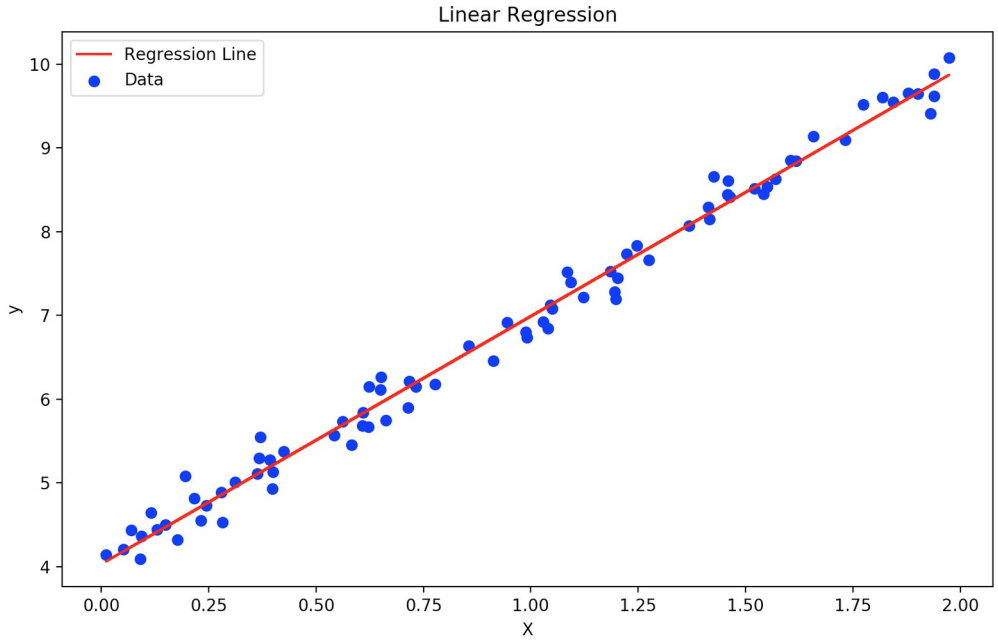


Linear Regression

Lectures — vi LinearReg_demo_for_screenshots.py — 107×29

```
# Train a Linear Regression model
import time
t1 = time.time()
model = LinearRegression(n_jobs=2)
model.fit(X_train, y_train)
print(time.time()-t1)

# Display the model parameters
print(f"Intercept: {model.intercept_[0
print(f"Coefficient: {model.coef_[0][0

# Visualize the fitted line
plt.figure(figsize=(10, 6))
plt.scatter(X_train, y_train, color='b
plt.plot(X_train, model.intercept_[0]+
plt.xlabel('X')
plt.ylabel('y')
plt.title('Linear Regression')
plt.legend()
plt.show()

# Make predictions on test data
y_pred = model.predict(X_test)

# Evaluate the model
mse = np.mean((y_test - y_pred) ** 2)
print(f"Mean Squared Error: {mse:.2f}")
```

Figure 1



Lectures — -bash — 88×9

```
(base) Deepaks-MacBook-Pro:Lectures deepak$ ./LinearReg_demo.py
0.0016701221466064453
Intercept: 4.03
Coefficient: 2.96
Mean Squared Error: 0.03
(base) Deepaks-MacBook-Pro:Lectures deepak$
```